



Data Cryptography Services (DCS) 3.7 Product Guide

Guides

Prepared By:

Electronic acknowledgment on file in Smartbear
Collaborator.

Ken G Gieselman
Author

Date

Approved By:

Electronic signature on file.

Michael Pajevski
Title

Date

Electronic signature on file.

Richard Cho
Document Release Authority

Date

Configuration Item: Data Cryptography Services – 681.2

MGSS Document No.: **DOC-005733**

Issue Date: May 14, 2026

Jet Propulsion Laboratory
California Institute of Technology

EPDM Electronic Signatures

User-Group/Role	...	Decision	Comments	Date
Pajevski, Michael J (pajevski)-JPL Consumer/...	...	Approve		14-May-2026 15:23

Reviewers

Review solicited from the following and includes the required minimum reviewers (in bold) as specified in DOC-000016 Documentation Structure, Standards and Definitions. The Review Date identifies respondents but does not indicate approval or concurrence, only acknowledgement that the reviewer has reviewed the document, submitted recommendations/feedback, and are aware of the document and its impact.

Review Date	Name/Title
12/02/2024	Eva Bokor S/W Quality Assurance Engineer (SQAE)
02/04/2025	Kam S Tso Cognizant Engineer (CogE)
05/11/2026	Ken G Gieselman Task Manager (TM)
03/04/2025	Kyran J Owen-Mankovich Systems Engineer Office Manager (SEOM)
05/06/2026	Michael J Pajevski SSE Application Security Subsystem (ASEC)
05/23/2025	Scott D Markham MITD Architect and Lead System Engineer

CHANGE LOG

Revision	Submission Date	Affected Sections or Pages	Change Summary
Initial	5/5/2026	All	Initial issue of document.

TABLE OF CONTENTS

1	DOCUMENT OVERVIEW.....	1
1.1	IDENTIFICATION.....	1
1.2	PURPOSE.....	1
1.3	TERMINOLOGY AND NOTATION	1
1.4	REFERENCES.....	1
1.5	PRODUCT OVERVIEW.....	1
2	PRODUCT REQUIREMENTS	3
2.1	OPERATING SYSTEM REQUIREMENTS	3
2.2	HARDWARE REQUIREMENTS	3
2.3	CRYPTOGRAPHIC ARTIFACT REQUIREMENTS	4
2.4	DCS CRYPTO CLIENT REQUIREMENTS	5
2.5	CRYPTO SERVICE REQUIREMENTS	5
2.6	DCS SDLS SERVICE REQUIREMENTS.....	6
2.7	DCS SA MANAGEMENT GUI SERVICE REQUIREMENTS	6
2.8	DCS SERVICE CONTAINER REQUIREMENTS	6
3	BUILDING AND PACKAGING DCS SOFTWARE	7
3.1	RETRIEVING AND PREPARING THE DCS SOURCE CODE	7
3.2	SOFTWARE DEPENDENCIES.....	7
3.3	CUSTOMIZING THE DCS BUILD	7
3.4	EXECUTING THE DCS BUILD	9
3.5	INTEGRATED TESTING	10
3.6	PACKAGING DCS	10
3.7	TROUBLESHOOTING & REPORTING BUILD ISSUES	12
4	DEPLOYMENT PREPARATION.....	13
4.1	PREPARE CRYPTOGRAPHIC ARTIFACTS	13
4.2	PREPARE CONFIGURATION INFORMATION	22
4.3	PREPARE SDLS SECURITY ASSOCIATIONS	24
5	DEPLOYMENT	29
5.1	DEPLOYMENT OVERVIEW	29
5.2	NON-CONTAINERIZED DEPLOYMENT	29
5.3	DCS SERVICE CONTAINER DEPLOYMENT	46
5.4	DEPLOYING THE SECURITY ASSOCIATION DATABASE (SADB).....	52
5.5	VALIDATING DCS DEPLOYMENTS.....	59
5.6	TROUBLESHOOTING DEPLOYMENT ISSUES	61

6	ADMINISTRATION	65
6.1	SECURITY ASSOCIATION (SA) MANAGEMENT	65
6.2	LOGGING	79
6.3	ENVIRONMENT VARIABLES	80
6.4	NETWORK INTERFACES	81
7	PRODUCT SUPPORT	82
7.1	DEFECT REPORTING PROCEDURE	82
7.2	POINTS OF CONTACT	82
	APPENDIX A - ABBREVIATIONS AND ACRONYMS	83
	APPENDIX B - EXAMPLE SA CREATION CSV FILE	85

TABLE OF TABLES

Table 1: Software Identification	1
Table 2: Applicable MGSS Documents.....	1
Table 3: SADB CSV Fields	25
Table 4: SADB Fields Not in CSV	26
Table 5: Environment Variables	81
Table 6: TCP/UDP Port Information	81
Table 7: Points of Contact.....	82

1 DOCUMENT OVERVIEW

1.1 IDENTIFICATION

Table 1: Software Identification

Property	Value
Configuration ID (CI)	681.2
Element	Systems Engineering Office (SEO)
Program Set	Data Cryptography Services (DCS) Formerly known as Key Management and Cryptography (KMC)
Version	3.7.x

1.2 PURPOSE

This document provides deployment and administration instructions for the DCS software.

1.3 TERMINOLOGY AND NOTATION

See Appendix A for definitions of the abbreviations and acronyms used in this document. Command examples are denoted by starting with the ‘\$’ character and being depicted in a monospaced font such as:

```
$ sestatus | grep Current
```

Other examples of configuration files, database commands or command/script output will also be denoted with the monospace font.

1.4 REFERENCES

Table 2: Referenced MGSS Documents

Title	Document Number
<i>DCS 3.7 Software Interface Specification (SIS)</i>	DOC-005776, Rev. A

1.5 PRODUCT OVERVIEW

DCS provides cryptography capabilities, which include encryption/decryption of files and creation/verification of integrity check values (ICVs) for files. DCS also includes command encryption capabilities via application of the Consultative Committee for Space Data Systems (CCSDS) “Blue Book” Space Data Link Security (SDLS) standard.

This guide describes how to deploy and administer the DCS software. See the DCS 3.7 Software Interface Specification (SIS) for information about using the DCS application programming interfaces (APIs).

There are four components in the DCS software:

- Crypto Client - Provides command-line interfaces (CLIs) for cryptographic operations, software libraries that provide Java APIs for cryptography, and C and Python APIs for command encryption/authentication.
- Crypto Service - REST APIs for performing cryptography.
- SDLS Service –REST APIs for using SDLS functionality.
- Security Association (SA) Management GUI –Web user interface for managing SDLS SAs.

Using the SDLS capabilities of DCS requires a Security Association Database (SADB) that holds the Security Associations defined in the CCSDS SDLS standard. The SADB in DCS utilizes a MariaDB server that can be configured on the same host as the Crypto Client, Crypto Service, SDLS Service, SA Management GUI, or on any other network-reachable system.

All the DCS components listed above can be deployed on the same host or on separate hosts. If the DCS components are installed on separate hosts, the DCS Crypto Client must be installed on systems hosting command and control software (e.g., AMMOS Instrument Toolkit (AIT)) if the SDLS C or Python APIs will be used. The DCS Crypto Client does not have to be installed on the same host(s) as the command and control software if that software will use the SDLS REST API.

When possible, software applications that use DCS for cryptography should be integrated with the DCS Crypto Service rather than the DCS Java APIs (i.e., local libraries). Using the Crypto Service provides a more secure solution because the cryptography and associated keys are handled by trusted services.

The DCS Crypto Service can be configured to use an external Key Management Service (KMS) to manage cryptographic keys as well as a local keystore file. The Crypto Service uses the Key Management Interoperability Protocol (KMIP) to interact with KMS services, and should be compatible with any KMS that provides a KMIP interface. Please note that the use of a KMS requires third-party software and/or services – i.e., *DCS does not include any KMS capabilities*.

DCS software development is carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (NASA) (80NM0018D0004), and in partnership with NASA Independent Verification and Validation (IV&V).

2 PRODUCT REQUIREMENTS

2.1 OPERATING SYSTEM REQUIREMENTS

DCS is developed and tested on the Red Hat Enterprise Linux (RHEL) 9 platform. This DCS release is intended for use on RHEL 9.4 Server, and was tested on that platform. DCS 3.7 **is not limited** to RHEL 9.4 – it should work on any RHEL 9.x Linux system or similar Linux variants, including Oracle Linux, Rocky Linux, Alma Linux, etc., as long as the software dependencies can be satisfied.

Of particular note: DCS 3.7.0 **was not tested** with the FIPS-140-3 encryption support mode on RHEL 9. The previous version of the software was verified to work properly with FIPS 140-2 support mode enabled on RHEL 8, and FIPS 140-3 support will be tested on a future version of DCS. To operate in a FIPS-enabled mode, DCS must be configured with Bouncy Castle FIPS Keystore (BCFKS)-formatted keystores. See Section 4.1.2.2 for instructions on how to generate BCFKS keystores and convert from other formats to BCFKS. To check the configuration status of the FIPS-140 module on RHEL 9, use:

```
$ fips-mode-setup --check
```

DCS 3.7 **will operate correctly** if SELinux is enabled and configured in “Enforcing” mode, as long as SELinux is configured to use the “targeted” policy. The DCS software packages include all necessary SELinux policy files to allow full operation – for files installed *in the DCS packages*. Additional SELinux policies/configuration may be needed to support software provided by the Operating System, additional capabilities including load balancers, KMS capabilities, etc, and to set appropriate SELinux contexts for artifacts such as TLS keys, certificates, keystores and truststores.

To check the configuration of SELinux on a RHEL 9 system, use:

```
$ sestatus
```

Should either FIPS mode or SELinux configuration need to be changed prior to DCS installation, please see the system documentation in the Red Hat Knowledgebase at <https://redhat.com>.

2.2 HARDWARE REQUIREMENTS

The hardware requirements for DCS 3.7 largely depend on the amount of cryptography that it will have to do and/or the amount of data to be handled. Light usage of the Crypto Client and DCS Services can run on a system with 1 or 2 modern x64 processor cores and 4 GB of memory. Heavier usage will most likely require more resources.

By default, the individual DCS Services are configured to use 2-3GB of Java Heap space for operating memory *per service*, though these can be adjusted for each service configured. Running the Crypto Client *and* all three DCS Services (Crypto Service, SDL Service, SA Management GUI) with the default settings will perform best on a system with at least 2 modern x64 processor cores and 8 GB of RAM available.

2.3 CRYPTOGRAPHIC ARTIFACT REQUIREMENTS

All the DCS components deal with cryptography, *but do not provide* any of the cryptographic artifacts – keys, certificates, keystores, truststores, etc. – that enable cryptography. There are a number of cryptographic artifacts that need to be created prior to any DCS deployment:

- **TLS Keys & Certificates** – these files provide the basic cryptographic elements for TLS encryption. TLS certificates contain information about the certificate holder, how the certificate may be used, the authority chain used to create the certificate, a time period during which the certificate is valid for use, etc. TLS keys are a private, sometimes passphrase-protected, key that can be used to “unlock” the certificate. TLS keypairs (key and certificate) are used by the DCS Components to encrypt communications between the client and the services, between the services, and between the services and external capabilities like a KMS. TLS keypairs can also be used for mutual TLS (mTLS) authentication, where a service or tool is configured to require the client to provide a “trusted” certificate that can be used to validate the connection. Generating TLS keypairs often involves several steps; see Section 4.1.1 for details on how to generate TLS keypairs for use with DCS.
- **TLS Keystores** – these files are used by Java-based tools and services to provide “secure” storage of TLS keypairs. TLS keystore files can be stored in several formats: Java Keystore Format (.jks), PKCS#12 (.p12), and Bouncy Castle FIPS Keystore (.bcfks), all of which are supported by the DCS components. Bouncy Castle FIPS Keystore formatted keystores are recommended for use with DCS, because they are usable in the widest range of configurations. TLS Keystores are usually generated from the TLS keys and certificates that they store; see Section 4.1.2 for instructions on how to generate the various keystore files and formats for use with DCS.
- **Crypto Keystore** – this Java keystore is one of the ways that cryptographic keys can be stored for use by the DCS Crypto Service and DCS Crypto Java APIs. This keystore file can be generated in the same formats as TLS keystores: Java Keystore Format (.jks), PKCS#12 (.p12), and Bouncy Castle FIPS Keystore (.bcfks). The Bouncy Castle FIPS Keystore (.bcfks) format is recommended for this keystore as well, to allow for the widest range of use. See Section 4.1.3 for details on how to create a Crypto Keystore.
- **TLS Truststores** – these files contain a list of certificates that are trusted to sign other TLS certificates. Most operating systems provide a default set that can be evaluated for use, but a more carefully curated list is recommended for best security practices. Many sites have an accepted or approved bundle that can be used. For sites that choose to utilize mutual TLS authentication, a special truststore should be constructed with specific certificates to trust. DCS requires TLS truststores in two formats: a Java truststore, which can be in any of the keystore formats listed above (JKS format preferred), and a x.509 “bundle” – a file containing the concatenated Certificate Authority (CA) certificates to be trusted. See Section 4.1.4 for details on how to generate the TLS Truststores.
- **mTLS Truststores** – these files contain a list of certificates that are trusted for mutual TLS authentication between the DCS Crypto Client and the DCS Services, or between the DCS Services. These truststores are only needed in a Java Keystore format – JKS, P12, or BCFKS. JKS format is recommended for easiest configuration. See Section 4.1.4 for details on how to configure a mTLS Truststore.

2.4 DCS CRYPTO CLIENT REQUIREMENTS

The DCS Crypto Client can use a Key Management Service (KMS) or a keystore file (in JKS, PKCS12, or BCFKS format) to get cryptographic keys. If a KMS is used, it must comply with the Key Management Interoperability Protocol (KMIP) standard with the TLSv1.2 authentication profile.

If the SDLS capabilities of the DCS Crypto Client are used, the libcrypt C library must be installed on the same host as the DCS Crypto Client and a MariaDB database is required. The MariaDB database can be located on the same host as the DCS Crypto Client or any other host accessible by the client. The SDLS capabilities of the DCS Crypto Client require the data-level cryptography capabilities of the DCS Crypto Service.

The Crypto Java APIs in the DCS Crypto Client write to log files. The installation instructions in Section 5.2.3.1 include changing the group of the DCS Crypto Client's logging directory or putting all users (of applications that use the DCS Crypto Client) into the default group ("users") on the DCS Crypto Client logging directory (default: `/opt/amos/kmc/logs`) and all log files in it. Prior to starting DCS software installation, determine which group will be used to provide write access to the DCS logging directory and log files.

2.5 CRYPTO SERVICE REQUIREMENTS

The DCS Crypto Service uses a KMS or a keystore file to get cryptographic keys. The keys are from the KMS or keystore based on the Key Reference (keyRef) specified in each request sent to the Crypto Service. If both KMS and keystore file are configured, the Crypto Service will attempt to retrieve the key from the keystore first, and then from the KMS.

The keys that will be used by the Crypto Service must be specified as part of the design of the system that uses DCS. See Appendix B for an example of a set of keys for a DCS use case.

DCS does not provide a KMS. DCS provides encryption & decryption (and ICV creation & verification) capabilities that can use the Key Management Interoperability Protocol (KMIP) standard to communicate with a KMS. A project, institution, multi-mission program, or other organization can (as allowed by the project's governing bodies) operate a KMS to be used by a project. Each project that uses DCS must determine which KMS will be used by the project. There are many commercial offerings of KMIP-compliant KMS products.

A PKCS#12, JKS, or BCFKS keystore file can be used instead of a KMS. BCFKS is required if DCS is running on a FIPS-enabled system. Each project is responsible for creating and managing their keystores. The instructions in this document cover creating and using keystore files, but guidance about the overall key management processes/procedures and policies applicable to flight projects is beyond the scope of this document.

By default, the DCS Crypto Service, SDLS Service, and SA Management GUI all use mutual TLS for authentication – but this approach allows all users the same level of access to the service. For more fine-grain access control, it is necessary to use a product like the AMMOS Common Access Manager (CAM), which is available to NASA missions but is not offered as open-source. If deploying DCS into an environment that uses AMMOS CAM for access control, the CAM Server that will be used must be available, and authorization policies created on that server prior to DCS deployment.

The DCS Crypto Service uses the `crypto` group, by default, to provide access to the Crypto Service logs. The `crypto` group is created on the local host if a group with that name does not already exist (on the local host or in LDAP). For systems that use LDAP for operating system users and groups, creating the `crypto` group in LDAP before installing the Crypto Service software facilitates managing `crypto` group membership in LDAP rather than on a host-by-host basis. For further details on customizing service users, groups, and permissions, see Section 3.3.

2.6 DCS SDLS SERVICE REQUIREMENTS

The SDLS Service requires cryptographic artifacts, configuration parameters, and a MariaDB database for the SADB.

Just like the Crypto Service, the SDLS Service is configured to use mTLS for authentication to the service, and runs as the `crypto` group.

2.7 DCS SA MANAGEMENT GUI SERVICE REQUIREMENTS

The SA Management GUI Service requires cryptographic artifacts, configuration parameters, and a MariaDB database for the SADB.

Just like the Crypto Service and SDLS Service, the SA Management GUI Service is configured to use mTLS for authentication to the service, and runs as the `crypto` group.

2.8 DCS SERVICE CONTAINER REQUIREMENTS

The DCS services are also available as Open Container Initiative (OCI) compliant container images based on the Red Hat 9 Universal Basic Image (UBI), which can be used with a variety of container hosting tools (Docker, Podman, AWS ECS, etc.).

To minimize the size and complexity of the container deployments, several of the deployment options for the DCS services have been preselected. Mutual TLS is used for access to the SADB and the DCS Services. The DCS Crypto Service Container is also configured to obtain keys from a BCFKS keystore, and cannot be configured in the 3.7.0 release to use a KMS.

Nearly all of the requirements for the DCS Services in the sections above are satisfied by the DCS Service Containers as delivered, with the following exceptions:

- **Crypto Keystore:** The DCS Crypto Service Container is configured to obtain cryptographic keys from a BCFKS keystore available at deployment time, in a base64 encoded file.
- **TLS Artifacts:** A PEM-encoded TLS key and certificate pair, keystore, truststore, and CA bundle must be provided for the hostname where the DCS services will reside. See Section 4.1 for details on how to generate these artifacts.
- **SADB FQDN:** the fully-qualified hostname of the SADB must be known for deployment.
- **Crypto Service FQDN:** The fully-qualified hostname of the Crypto Service Container must be known at deployment time. This can be a DNS alias for the Crypto Service Container's assigned IP established after the deployment for dynamic environments like AWS ECS.
- **Container User Namespace Mapping Offset:** Containerization systems like Docker and Podman offer a User Namespace Mapping option that maps UIDs and GIDs within the container to a specific range of UIDs and GIDs on the host system, to provide some separation between users inside and outside the container. The offset values must be known at deployment time. See Section 5.3.1 for instructions about namespace mapping.

3 BUILDING AND PACKAGING DCS SOFTWARE

DCS is distributed as an open-source software project, subject to the Apache 2.0 software license. The primary repository for the code is located on Github.com:

<https://github.com/NASA-AMMOS/DCS>

3.1 RETRIEVING AND PREPARING THE DCS SOURCE CODE

The DCS source code can be retrieved with a variety of tools that allow interaction with Git repositories – all the processes and examples provided below will use the basic git command line client. The following steps will retrieve the DCS Source Code, and pull in any submodules necessary, like NASA CryptoLib. These steps require no special privileges.

```
$ git clone https://github.com/NASA-AMMOS/DCS
$ cd DCS
$ git submodule init; git submodule update
```

3.2 SOFTWARE DEPENDENCIES

DCS requires several software dependencies in order to build and execute unit tests. Required software can be installed with OS Vendor packages, pulled directly from public repositories, or any other method. Make sure the required tool is available in the build environment appropriately for that particular tool (PATH for executables, LD_LIBRARY_PATH for shared libraries, PYTHONPATH for python modules, etc.). Tool versions listed outside of square brackets are minimum versions, version within square brackets specify the tool/version used for the official DCS build and testing.

- C Compiler [gcc 11.4.1]
- Java Development Kit 17.x [OpenJDK 17.0.13]
- cmake [3.26.5]
- swig [4.0.2]
- MariaDB Connector/C [3.2.6]
- Python Interpreter [3.9.19]
- cffi (Python module) [1.14.5]
- invoke (Python module) [2.2.0]
- pycryptodome (Python module) [3.20.0]
- maven [3.6.3]

3.3 CUSTOMIZING THE DCS BUILD

The DCS build, test, and deployment scripts all use the same environment configuration file in order to keep the various operations in sync. That file is distributed in the DCS source code, and exists at <kmc_src_root>/kmc-resources/scripts/setenv.sh.

```
VERSION=3.7.0

# Deployment Settings
CRYPTO_SERVICE_FQDN="crypto.example.com"
SADB_FQDN="sadb.example.com"

# Note: Installation locations other than /opt/ammos/kmc are not supported in 3.7.0.
PREFIX="/opt/ammos/kmc"
```

```

BINPATH="${PREFIX}/bin"
INCPATH="${PREFIX}/include"
export LIBPATH="${PREFIX}/lib"
CFGPATH="${PREFIX}/etc"
LOGPATH="${PREFIX}/logs"
TESTPATH="${PREFIX}/test"
SVCPATH="${PREFIX}/services"
DOCPATH="${PREFIX}/docs"

CRYPTOSVC_PREFIX="${SVCPATH}/crypto-service"
CRYPTOSVC_LIBPATH="${CRYPTOSVC_PREFIX}/lib"
CRYPTOSVC_CFGPATH="${CRYPTOSVC_PREFIX}/etc"
CRYPTOSVC_LOGPATH="${CRYPTOSVC_PREFIX}/logs"

SDLSSVC_PREFIX="${SVCPATH}/sdls-service"
SDLSSVC_LIBPATH="${SDLSSVC_PREFIX}/lib"
SDLSSVC_CFGPATH="${SDLSSVC_PREFIX}/etc"
SDLSSVC_LOGPATH="${SDLSSVC_PREFIX}/logs"

SAMGMTSVC_PREFIX="${SVCPATH}/sa-mgmt"
SAMGMTSVC_LIBPATH="${SAMGMTSVC_PREFIX}/lib"
SAMGMTSVC_CFGPATH="${SAMGMTSVC_PREFIX}/etc"
SAMGMTSVC_LOGPATH="${SAMGMTSVC_PREFIX}/logs"

DEPLOY_KMC_CRYPTO_SERVICE=0
DEPLOY_KMC_SDLS_SERVICE=0
DEPLOY_KMC_SA_MGMT_SERVICE=0

SERVICE_JRE=/usr/lib/jvm/jre-17-openjdk

KMC_CRYPTO_SERVICE_PORT=8443
KMC_SDLS_SERVICE_PORT=8445
KMC_SAMGMT_SERVICE_PORT=8447

# Deployment Users/Groups
# Crypto Group
CRYPTOGRP_NAME="crypto"
CRYPTOGRP_GID=59111
# cryptogrp_members is a comma separated list of users to verify membership
CRYPTOGRP_MEMBERS=""

# Crypto User
CRYPTOUSR_NAME="crypto"
CRYPTOUSR_UID=59111
CRYPTOUSR_HOME="${PREFIX}"
CRYPTOUSR_SHELL="/sbin/nologin"
CRYPTOUSR_COMMENT="KMC Service User"

# Config Group
CFGGRP_NAME="kmccfg"
CFGGRP_GID=59105
# cfggrp_members is a comma separated list of users to verify membership
CFGGRP_MEMBERS=""

# Config User
CFGUSR_NAME="kmccfg"
CFGUSR_UID=59105
CFGUSR_HOME="${PREFIX}"
CFGUSR_SHELL="/sbin/nologin"
CFGUSR_COMMENT="KMC Configuration User"

```

```

# Users Group - for CLI logging
USERSGRP_NAME="users"
USERSGRP_GID=100
# usersgrp_members is a comma separated list of users to verify membership
USERSGRP_MEMBERS=""

# Build Settings
if [ -z "${JAVA_HOME}"; then
    export JAVA_HOME=/lib/jvm/java-17-openjdk
fi
PYTHON_INT=/usr/bin/python
PYTHON_VER=`${PYTHON_INT} -V | /bin/cut -f2 -d' ' | /bin/cut -f1-2 -d'.'`
PYTHONPATH="${LIBPATH}/python${PYTHON_VER}/site-packages"

# Container settings
# CONTAINER_TYPE can be podman or docker, depending on which is installed
#     locally to produce images
CONTAINER_TYPE="podman"

# Do not edit settings below -- needed for build steps
pushd $(dirname "$0")
ROOT=$(git rev-parse --show-toplevel)
DIST=$ROOT
BUILD_DIR=$DIST/build$BUILD/build
RAW_DIR=$DIST/build$BUILD/raw
SRC_KMC=$DIST
SRC_KMIP=$DIST/kmip-client
SRC_CRYPTOLIB=$DIST/amos-cryptolib
SRC_RSC=$SRC_KMC/kmc-resources

```

Figure 1: Default DCS setenv.sh build configuration file

Most of the settings in the build configuration file can be altered to suit intended deployment patterns. By default, DCS installs in `/opt/amos/kmc`, with a few files in `/lib/systemd/system` and `/etc/sysconfig` if the DCS services are deployed. Those paths can all be altered with the various PATH settings in the `setenv.sh` file. Any configuration changes *should be completed prior to executing the DCS Build*, as settings changed may alter the final build artifacts.

Pay close attention to all of the PATH options that define where various pieces of DCS will be installed on the filesystem and the Users and Groups defined for the different permissions boundaries that are needed.

Please note that for DCS 3.7.0, `PREFIX` in the `setenv.sh` above should remain set to `"/opt/amos/kmc"`. If it is changed, some software features may not work properly.

The users and groups needed for DCS deployment and operation can be created locally on each system or in a LDAP or other network-based directory service configured for use on the system. Package-based deployments can create users and groups as needed locally, if they're not already defined at package installation.

3.4 EXECUTING THE DCS BUILD

Building the DCS source code can be accomplished with one script. This script should be executed in the root of the DCS Source Code distribution, and requires no special privileges.

```
$ kmc-resources/scripts/build.sh
```

Executing the build script with the default configuration compiles all of the necessary code and executes the available unit tests. If problems are encountered during the build execution, please see Section 3.7 for additional details on diagnosing build problems and how to reach out for assistance.

3.5 INTEGRATED TESTING

All of the integrated unit tests are executed by the build script. Results from each set are reported in the script output, similar to:

```
[INFO] Running gov.nasa.jpl.ammos.kmc.crypto.library.test.DigitalSignatureTest
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 26.45 s -- in
gov.nasa.jpl.ammos.kmc.crypto.library.test.DigitalSignatureTest
```

In general, none of these tests should fail. If a test does fail, it's worthwhile trying to determine why. Please see Section 3.7 below for more information on diagnosing test failures and how to reach out for assistance.

3.6 PACKAGING DCS

Starting in DCS 3.7, DCS can be packaged for delivery in many different ways. DCS can be deployed as bare files on a system, via an archive (tar, zip, etc), via RPM packages. The DCS Services (Crypto Service, SDLS Service, and SA Management GUI) can be deployed as standalone OCI-compliant container images. When possible, the use of either OS-supported packages (RPMs for RHEL-based Linux systems) or the containerized service images are strongly recommended for long-term software management. All of the various DCS packaging forms can be built using the `deploy.sh` script (in `<kmc_src_root>/kmc-resources/scripts/`).

Usage of the `deploy.sh` script is (available with the `-h` and/or `--help` flags):

```
Usage: kmc-resources/scripts/deploy.sh
[ -a | --all-services ] - deploy ALL KMC services
[ -c | --crypto-service ] - deploy KMC Crypto Service
[ -h | --help ] - display deploy.sh script usage
[ -i | --img ] - produce OCI container images for KMC Services
[ -m | --sa-mgmt-service ] - deploy KMC SA Management Service
[ -s | --sdls-service ] - deploy KMC SDLS REST API
[ -p DIR | --pkg DIR ] - deploy for package building in DIR
[ -r | --rpm ] - deploy and build RPM packages in kmc-resources/packaging/rpm
[ -t DIR | --tar DIR ] - deploy/build tar.bz2 archive in DIR
[ -z DIR | --zip DIR ] - deploy/build zip archive in DIR
```

3.6.1 ARCHIVE FILES

The `deploy.sh` script can produce archive files for later deployment or deployment on other systems, in either Tar/Bzip (.tar.bz2) or Zip (.zip) formats. Please note – in order to make sure that all of the file and directory owner, group, and permissions settings are correct, **using `deploy.sh` to generate an archive file should be performed as a privileged user** (root, via sudo, etc.). If the archive file is created by an unprivileged user, all of the files and directories would be owned by the creating user.

To produce an archive, use any of the `-t`, `--tar`, `-z` or `--zip` option flags, followed by a directory:

```
$ kmc-resources/scripts/deploy.sh --tar /tmp/kmc
```


The `deply.sh` script will create a full deployment of DCS in the `/tmp/kmc` directory, then create a Tar/Bzip2 archive file in that directory with the deployed components (`/tmp/kmc/kmc-3.7.0.tar.bz2`), and delete the deployed files with the exception of the archive.

Replacing `--tar` with `--zip` (or `-z`) will produce a Zip archive in that directory (`/tmp/kmc/kmc-3.7.0.zip`) instead of a Tar/Bzip2 archive.

3.6.2 RPM PACKAGES

The `deploy.sh` script can also create RPM packages, for use on RHEL-based Linux systems. This *does not* require elevated privileges to do, unlike archive creation – the RPM packaging handles owner, group and permission settings in a completely different manner. Creation of RPM packages takes place in `<kmc_src_root>/kmc-resources/packaging/rpm`, utilizing the RPM Specification in `<kmc_src_root>/kmc-resources/packaging/rpm/SPECS/kmc.spec`. The RPMs produced by the process can be found in `<kmc_src_root>/kmc-resources/packaging/rpm/RPMS/x86_64`. The RPM specification included with the DCS Source Code produces four independent RPM packages that may be used together or separately. The RPMs produced are:

```
kmc-3.7.0-1.el9.x86_64.rpm
kmc-crypto-service-3.7.0-1.el9.x86_64.rpm
kmc-sa-mgmt-service-3.7.0-1.el9.x86_64.rpm
kmc-sdls-service-3.7.0-1.el9.x86_64.rpm
```

The main "kmc" package provides the Crypto Client component. The other three packages each provide one of the DCS Services, including configuration and SystemD unit files for each. The RPM packages can be installed with OS-provided tools like `dnf` and `rpm`. Use of a package manager like `dnf` or `yum` and package repositories is highly recommended for long term software maintenance:

```
$ sudo dnf install kmc-3.7.0-1.el9.x86_64.rpm
$ sudo rpm -ivH kmc-3.7.0-1.el9.x86_64.rpm
```

The service packages include supporting files for configuration and automated service management (SystemD Unit files). See Section 5.2.3.2 for details on configuring and managing the DCS Services once deployed.

3.6.3 CONTAINERS

The `deploy.sh` script can also generate OCI-compliant container images for the DCS services that are usable with Docker, Podman, and Amazon Web Services (AWS) Elastic Cloud Service (ECS). The DCS images are likely compatible with any container hosting platform that supports OCI images, but the platforms listed above are the only ones that have been tested.

The container runtime tool used to build the container images is specified by the `CONTAINER_TYPE` parameter in `<kmc_src_root>/kmc-resources/scripts/setenv.sh`:

```
CONTAINER_TYPE="podman"
```

The images are built based on the Dockerfiles in `<kmc_src_root>/kmc-resources/packaging/container/<service>`. Once built, the images will be available in the local container image storage system, tagged with the name and version.

Invoking `deploy.sh` with the `-i` or `--img` flags will create the container images on the build system, using the container engine specified in the `CONTAINER_TYPE` variable in

```
<kmc_src_root>/kmc-resources/scripts/setenv.sh:
```

```
$ kmc-resources/scripts/deploy.sh --img
```

No additional arguments are necessary to include the DCS Services. The generation of the container images will package all three DCS Services (`crypto-service`, `sdls-service`, `sa-mgmt-service`) by default. This use of the `deploy.sh` script *does not* require elevated privileges – as long as the container engine is configured and allows the current user to build and interact with container images.

Images in the podman repository (or docker, if that tool was configured in `CONTAINER_TYPE` in `<kmc_src_root>/kmc-resources/scripts/setenv.sh`) can be saved to a transportable file with the following command:

```
$ podman image save -o <output_file_name> <image_name>:<image_version>
```

So for DCS 3.7.0, the images can be saved to files with:

```
$ podman image save -o kmc-crypto-service-3.7.0.tar kmc-crypto-service:3.7.0
$ podman image save -o kmc-sa-mgmt-service-3.7.0.tar kmc-sa-mgmt-service:3.7.0
$ podman image save -o kmc-sdls-service-3.7.0.tar kmc-sdls-service:3.7.0
```

For AWS ECR or other network-based container image repositories, the process usually involves tagging the local image with the location of the remote repository, authenticating to the remote repository, and then pushing the image across. See the documentation for the desired remote repository for exact steps to import a container image in this fashion.

3.7 TROUBLESHOOTING & REPORTING BUILD ISSUES

DCS build and test issues usually fall into one of the following categories:

- Missing dependency – one or more of the build dependencies could not be located on the build system or retrieved from public repositories. Check the output from the build script and try to resolve the missing dependencies.
- Configuration issue – for some reason, the configuration choices in `setenv.sh` are not working. Issues of this sort should be very rare, unless the settings somehow conflict (multiple services using the same directory, or a user/group setting that cannot be set, etc.).
- Tool issue – some build tools may not be 100% compatible with some platforms, or not be available at all. These issues can largely be avoided by using DCS on tested platforms – RHEL 9 on x86_64 for DCS 3.7.

Of course, none of those rule out actual code issues. As noted earlier in Section 2.1, Red Hat Enterprise Linux 9 on x86_64 was the only platform formally tested by the ASEC Team. Building DCS on other platforms may require some investigation if problems crop up with system includes, library compatibility, availability of software dependencies, and similar problems.

Issues and questions can be addressed by emailing the ASEC Team using the contact information in Section 7.2. Please review the set of existing issues at <https://github.com/NASA-AMMOS/DCS/issues> to determine if an issue has already been documented and the status of the known issues. Contributors may create issues directly, but the preferred method is to contact the ASEC team to discuss issues, and the ASEC team will manage the issues in GitHub.

4 DEPLOYMENT PREPARATION

Preparing for DCS software deployment involves:

1. Preparing cryptographic artifacts – see Section 4.1
2. Preparing the Security Associations (Sas) – see Section 4.2.4
3. Preparing configuration settings – see Section 4.2

4.1 PREPARE CRYPTOGRAPHIC ARTIFACTS

All of the DCS components utilize cryptography in a variety of ways, but do not provide any of the cryptographic artifacts – keys, certificates, keystores, truststores, etc. – that enable cryptography. Section 2.3 provides an overview of the necessary cryptographic artifacts. The subsections below describe how to create the artifacts.

At a minimum, make sure that the following artifacts have been created prior to moving forward with DCS deployment:

- TLS Server Key & Certificate for each server that will host the DCS Service components or the SADB (MariaDB) – see Section 4.1.1.1
- mTLS Client Keys & Certificates, if using mutual TLS authentication – see Section 4.1.1.2.
- Java Keystores for all TLS Keys & Certificates generated (Note: BCFKS-formatted keystores are recommended) – see Section 4.1.2
- Crypto Keystore, unless planning on solely using a KMS – see Section 4.1.3
- TLS Truststores, both in a Java keystore format (BCFKS recommended) and an X509 trust bundle – see Section 4.1.4
- mTLS Truststore, in a Java keystore format (BCFKS recommended), if using mutual TLS authentication – see Section 4.1.5

4.1.1 PREPARE TLS KEYS AND CERTIFICATES

The DCS software components use Transport Layer Security (TLS) key and certificate pairs to encrypt communication links, and may also use them to authenticate access to various services.

There are two sets of TLS keys and certificates that are used by the DCS software:

1. TLS Keys and Certificates for DCS Services (see Section 4.1.1.1)
2. TLS Keys and Certificates for mTLS Clients (see Section 4.1.1.2)

4.1.1.1 Prepare TLS Server Keys and Certificates

The following DCS services need key/certificate pairs:

1. Crypto Service
2. SDLS Service
3. Security Association Management GUI
4. MariaDB SADB

For *each server* that will host one or more of the services noted above, create a TLS key and certificate pair with a Common Name (CN) element specifying the fully-qualified domain name (FQDN) of the server.

In most cases, the process of creating a key and certificate pair starts by a system administrator creating a server key file (amos-server-key.pem) and a certificate signing request (.csr) file by using the following command. The expiration time of the generated certificate signing request and resulting certificate can be shortened or extended by changing the days parameter in the command – see the Certificate Authority’s documentation for minimum and maximum Certificate lifetimes supported.

```
$ openssl req -newkey rsa:2048 -days 365 -nodes \
    -keyout amos-server-key.pem -out <hostname>-cert-req.csr
```

Where <hostname> is the short name of the host.

Alternatively, the command above can be run with an option to use a config file (i.e., to specify values rather than be prompted), as follows (all one command):

```
$ openssl req -newkey rsa:2048 -days 365 -nodes \
    -keyout amos-server-key.pem -out <hostname>-cert-req.csr \
    -config cert.conf
```

An example cert.conf file is:

```
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
prompt = no
req_extensions = req_ext

[ req_distinguished_name ]
C = US
ST = California
L = Pasadena
O = NASA
OU = ProjextXYZ
CN = demo-kmc.example.com
emailAddress = "admin.list@example.com"

[ req_ext ]
subjectAltName = @alt_names

[alt_names]
DNS.1 = demo-kmc.example.com
```

To use a config file, create one with the correct the values for your system, and then specify it in the openssl command noted above.

The system administrator submits the certificate signing request to a certificate authority (CA) in order to obtain a server certificate file (to be named amos-server-cert.pem) from the CA.

The amos-server-cert.pem file should include the full certificate chain from the server cert to the root CA certificate. For example:

```
subject=/C=US/ST=California/L=Pasadena/O=Example/OU=IT/CN=kmc.example.com
issuer=/DC=gov/DC=nasa/DC=jpl/CN=JPLICA Issuing 1
-----BEGIN CERTIFICATE-----
MIIG5TCCBc2gAwIBAgITUwAAKo4ev88rsh2AygAAAAQjjANBgkqhkiG9w0BAQsF
[SNIP...]
```

```

UCtFgBPgyuISet//LUFpX8Ys5CsXOEuvRWPxIbTm2ftS4Hh30fZscIM=
-----END CERTIFICATE-----
subject=/DC=gov/DC=nasa/DC=jpl/CN=JPLICA Issuing 1
issuer=/CN=JPLICA Root
-----BEGIN CERTIFICATE-----
MIIEKDCCAxYgAwIBAgITPgAAAAIpWeePlZ5cAAAAAajANBgkqhkiG9w0BAQsF
[SNIP...]
vrufFb+w8AUV+1jcppRSIHsfTts8Jn4u
-----END CERTIFICATE-----
subject=/CN=JPLICA Root
issuer=/CN=JPLICA Root
-----BEGIN CERTIFICATE-----
MIIDBzCCAe+gAwIBAgIQWv9RtOxl8Y9Jdwc40lMSWTANBgkqhkiG9w0BAQsFADAW
[SNIP...]
Lcw=
-----END CERTIFICATE-----

```

Note that the subject and issuer lines are just comments, but they provide human-readable identification for each certificate. Your file may or may not have the same comments indicated above and may have other information in its content. The important content is the encoded data between the Begin Certificate and End Certificate lines. If the file issued from the Certificate Authority (CA) only contains the first certificate in the chain, the intermediate and root certificates from the Certificate Authority can be added into `amos-server-cert.pem` to have the full chain present.

4.1.1.2 Prepare mTLS Client Keys and Certificates

Mutual Transport Layer Security (mTLS) authentication can be used between the Crypto Client and DCS Services when an access control mechanism like the AMMOS Common Access Manager (CAM) is not used.

The mTLS authentication used by DCS requires a client key and certificate (for a group of users, or for each individual user -- depending on which option the project chooses).

The TLS server key/certificate pair(s) used by the DCS Crypto Service and/or MariaDB SADB should NOT be used as a client key/certificate pair. To avoid conflicts with the TLS server key, one option is to specify the CN in a `<user>-<hostname>.domain` format. For example, if all users will share a key/certificate pair (with group-based access on the key file), a CN like the following could be used:

```
client-demo-kmc.example.com
```

If different keys will be used for different (sets of) users, replace the "client" portion above with some suitable name for each key/certificate pair. To separate normal user access from admin level access to the SADB, create a separate certificate for admin users, such as:

```
admin-demo-kmc.example.com
```

The `openssl` command, and option to specify a config file, are the same for client key/certificate pairs as server key/certificate pairs described in the previous subsection – except the key and CSR file names are different, and the CN is different.

For example (non-admin cert):

```
$ openssl req -newkey rsa:2048 -days 365 -nodes -keyout ammos-client-key.pem \
-out client-cert-req.csr
```

For example (admin cert):

```
$ openssl req -newkey rsa:2048 -days 365 -nodes -keyout ammos-admin-key.pem \
-out admin-cert-req.csr
```

Use the CSR(s) to obtain the certificate(s) from an approved CA. Name the certificate(s) to properly distinguish them, such as `ammos-client-cert.pem` (for non-admins) or `ammos-admin-cert.pem` (for admin user certs).

4.1.2 PREPARE TLS KEYSTORES

Many of the DCS software components are Java-based and need TLS keystores in order to access key and certificate data. These keystores are generated from the key and certificate pairs generated in Section 4.1.1 in multiple keystore file formats. Choosing which keystore format to use is largely up to individual operators, though the use of the Bouncy Castle FIPS Keystore format is recommended with DCS, as it is the most widely usable of the three supported formats. Each of the TLS key and certificate pairs generated above need matching TLS Keystore files. Instructions for converting a TLS keypair to each of the keystore formats is provided below, and can be applied to all the TLS keypairs generated in Section 4.1.1.

4.1.2.1 Creating a PKCS#12 TLS Keystore

A PKCS#12 Keystore can be created from the key and certificate files generated in Section 4.1.1 can be accomplished with a single `openssl` command. Prior to running the conversion command, ensure that the certificate file contains the primary, all intermediate, and root certificates to provide a full authority chain. This example is for converting the `ammos-server-cert.pem` and `ammos-server-key.pem` to a new `ammos-server-keystore.p12`, but the filenames can be substituted for all of the TLS keypairs generated above.

```
$ openssl pkcs12 -export -in ammos-server-cert.pem \
-inkey ammos-server-key.pem -out ammos-server-keystore.p12
```

When prompted for the “export password”, enter the following: `changeit` (“changeit” is used throughout this document as *an example only*, and should be replaced by a strong password.)

4.1.2.2 Creating a BCFKS TLS Keystore

Generation of BCFKS keystores may require a local copy of the `bc-fips` Java library, if one isn’t already configured for system-wide Java use. A local copy can be obtained from the Maven Central repositories at the following URI (1.0.2.5 is the latest available at the time of this release, other versions may be substituted as desired):

<https://repo.maven.apache.org/maven2/org/bouncycastle/bc-fips/1.0.2.5/bc-fips-1.0.2.5.jar>

Generating a BCFKS-formatted keystore is best accomplished by using `keytool` to convert the PKCS#12 keystore generated in the previous subsection. This example, like the previous one, is for converting the `ammos-server-keystore.p12` into `ammos-server-keystore.bcfks`, but filenames can be substituted for all of the TLS keypairs and PKCS#12 keystores generated above. The `-J-Dcom.redhat.fips=false` argument may be necessary to read the PKCS#12 keystore on a RHEL-based system with the operating system FIPS mode *enabled*, and should not cause issues if the operating system FIPS mode is *disabled*.

```
$ /bin/keytool -J-Dcom.redhat.fips=false \
  -providerclass org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider \
  -providerpath /var/tmp/bc-fips-1.0.2.5.jar -importkeystore \
  -srckeystore ammos-server-keystore.p12 -srcstoretype PKCS12 \
  -srcstorepass changeit -deststoretype BCFKS -deststorepass changeit \
  -destkeypass changeit -destkeystore ammos-server-keystore.bcfks
```

Note: BCFKS-formatted keystores require that each key also has a key password assigned. This example uses the same password for the server key that it does for the keystore (changeit). As before, this password is only used as *an example only*, and should be replaced by a strong password in any DCS deployments.

4.1.2.3 Creating a JKS TLS Keystore

Use of the JKS format is not recommended, due to the age of the standard and generally weak cryptographic techniques used – DCS deployments should use BCFKS or PKCS#12 instead. Instructions for generating a JKS TLS Keystore are provided here for completeness. Generating a JKS-formatted keystore is best accomplished by using keytool to convert the PKCS#12 keystore generated in the Section 4.1.2.1. This example, like the previous two, is for converting the ammos-server-keystore.p12 into ammos-server-keystore.jks, but filenames can be substituted for all of the TLS keypairs and PKCS#12 keystores generated above. The -J-Dcom.redhat.fips=false argument may be necessary to read the PKCS#12 keystore and generate a JKS keystore on a system with the operating system FIPS mode enabled, and should not cause issues if the operating system FIPS mode is disabled.

```
$ /bin/keytool -importkeystore -srckeystore ammos-server-keystore.p12 \
  -srcstoretype PKCS12 -srcstorepass changeit -deststorepass changeit \
  -destkeypass changeit -destkeystore ammos-server-keystore.jks \
  -J-Dcom.redhat.fips=false
```

When prompted for the keystore password, enter the password for the keystore. Then, when prompted for the key password, just hit Enter (without entering a password).

4.1.3 PREPARING A CRYPTO KEYSTORE

To use the DCS Crypto Client or DCS Crypto Service with a Crypto Keystore (instead of a KMS), the keystore must be created. When DCS SDLS capabilities are used with a Crypto Keystore, it resides on the host(s) that will run the DCS Crypto Service.

The DCS Crypto Service package includes the following script to support creation of Crypto Keystore files from keying material in hex format, assuming the default DCS installation path:

- /opt/ammos/kmc/bin/create-crypto-keystore.sh

The script noted above has the following usage:

```
$ ./create-crypto-keystore.sh [-v] keystoreFile keystoreType keystorePasswordFile
  datafile [keyPasswordFile]
```

The `keystoreFile` is the path (relative or full) to the keystore file to create.

The `keystoreType` defines the type of keystore to create, and must be provided when invoking the script. It accepts one of two values: PKCS12 or BCFKS (Bouncy Castle FIPS Keystore).

The `keystorePasswordFile` is the path to a file containing the password to set on the keystore.

The `dataFile` argument refers to a file that contains one or more lines in the following format:

```
keyRef keyAlgorithm keyMaterial
# keyRef is the ID of the key, keyAlgorithm is AES (for any AES mode),
# and keyMaterial is the keying material in hex format.
```

The `keyPasswordFile` is an optional input, containing the password to use for individual keys within the generated keystore. If this input is not provided, then the keys within the keystore will not have individual passwords.

An example of running the command is (all on one line):

```
$ ./create-crypto-keystore.sh /tmp/crypto-keys.p12 PKCS12 /path/to/keystore_pass
/path/to/dataFile
```

An example of a keying material data file is shown below:

```
# Data for creating keys for CmdEnc testing
# format: keyRef keyAlgorithm keyMaterial_Hex
# NIST gcmtestvectors/gcmEncryptExtIV256.rsp
kmc/test/NIST0 AES ef9f9284cf599eac3b119905a7d18851e7e374cf63aea04358586b0f757670f8
kmc/test/NIST1 AES e9ccd6eef27f740d1d5c70b187734e11e76a8ac0ad1702ff02180c5c1c9e5399
kmc/test/NIST2 AES 7ecc9dcb3d5b413cad3af7b7812758bd869295f8aaf611ba9935de76bd87013
kmc/test/NIST3 AES a881373e248615e3d6576f5a5fb68883515ae72d6a2938e3a6f0b8dcb639c9c0
kmc/test/NIST4 AES 84c90349539c2a7989cb24dfae5e4182382ae94ba717d385977017f74f0d87d6
kmc/test/NIST5 AES 02803be2d88c804775b173be3b23bb49236ea3fc08b6170c7ce5f2afe11a84ff

# NIST cmactestvectors/CMACGenAES256.rsp
kmc/test/NIST_CMACE_90 AES
8a981033d7b4eb4cc9648ba26ad9679c1550dcf32df17fb80ef6b299440f7560
kmc/test/NIST_CMACE_91 AES
1bbf5a21762dd5a9371ea48fc6e919f494cdc3325d27dbb8aae8de1599477c52
kmc/test/NIST_CMACE_92 AES
f81ea615e48c0a25410ff15c1566b10a83fb9d839b667dc577333b5b83e5df81
kmc/test/NIST_CMACE_93 AES
23e5422e8d7560a9e65642b5e723a47536c16791f3a0cf918d3dee8adbec60fd
kmc/test/NIST_CMACE_94 AES
7cfc086d10659d7cb9247208358dd82c03b8dbd823323231df218e2448f4a79
kmc/test/NIST_CMACE_95 AES
d0f46fb37d516cc957aaefd3be2a8bede885330a8edb96f3e5e0ab8cd03a8c59
```

Projects should use a sufficiently strong, FIPS 140-2 or 140-3 certified, key generation tool – such as a commercial product for a Key Management Service (KMS) or another cryptographic key generator tool. In the development phase, a simple tool like `keytool` can be used as follows to generate a key and add it to a keystore file (creating the keystore file if it does not already exist).

4.1.3.1 Generating a BCFKS Crypto Keystore

Generation of BCFKS keystores may require a local copy of the `bc-fips` Java library, if one isn't already configured for system-wide Java use. A local copy can be obtained from the Maven Central repositories at the following URI (1.0.2.5 is the latest available at the time of this release, other versions may be substituted as desired):

<https://repo.maven.apache.org/maven2/org/bouncycastle/bc-fips/1.0.2.5/bc-fips-1.0.2.5.jar>

```
$ /bin/keytool -genseckey -alias test/key1 -keyalg AES -keysize 256 -storetype bcfks \
-keystore test-keystore.bcfks -providerpath bc-fips-1.0.2.5.jar \
-providerclass org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider
```


When prompted for the keystore password (twice), enter the password for the keystore. BCFKS keystores require that passwords be set for each of the keys, as well as for the keystore. Enter the desired key password when prompted (twice) or enter Return to use the keystore password as the key password.

4.1.3.2 Generating a PKCS#12 Crypto Keystore

```
$ /bin/keytool -genseckey -alias projxyz/key1 -keyalg AES -keysize 256 \
  -storetype pkcs12 -keystore crypto-keystore.p12
```

When prompted for the keystore password (twice), enter the password for the keystore.

4.1.3.3 Generating a JKS Crypto Keystore

Use of the JKS format is not recommended, due to the age of the standard and generally weak cryptographic techniques used – DCS deployments should use BCFKS or PKCS#12 instead. Instructions on generating the JKS Crypto Keystore are provided here for completeness.

```
$ /bin/keytool -genseckey -alias projxyz/key1 -keyalg AES -keysize 256 \
  -storetype jceks -keystore crypto-keystore.jks
```

When prompted for the keystore password, enter the password for the keystore. Then, when prompted for the key password, just hit Enter (without entering a password).

Note: Each key must have a unique alias (which DCS uses for keyRef). If same alias is used again, the key entry for that alias will be overwritten.

4.1.4 PREPARING TLS TRUSTSTORES

The DCS software components depend on TLS encryption to provide secure connections between the client and the various services. Secure TLS connections require validation of the certificates presented by the client and servers when connections are established. This is done by using a TLS Truststore – a collection of Certificate Authority (CA) certificates that are “trusted” to issue and sign the TLS certificates presented by the clients and servers.

This trust concept means that there is no one true list of CA certificates that is appropriate for every use case – rather, it is the project’s responsibility to build, maintain, and carefully evaluate the Certificate Authorities whose certificates should be trusted. The best security practice is to limit the trusted CAs to a minimal list – common practice, on the other hand, is to accept a wide range of governmental and commercial CAs that commonly issue certificates. Most operating system providers, Red Hat included, provide a set of default or system trust bundles that can be used to accept a wide range of certificates. Many organizations have their own curated lists that contain both internal and external CAs.

DCS uses two trust bundle formats in operation – a trust bundle of X.509-formatted CA certificates, and a Java Truststore with the same CA certificates, which can be BCFKS, PKCS#12, or JKS-formatted. The processes for creating and adding entries to both files are described in the following two subsections.

4.1.4.1 X.509 Trust Bundle

The X.509 trust bundle is a simple ASCII file containing a concatenated list of the CA certificates to be trusted. An example of an X.509 trust bundle is below:

```
subject=/C=US/O=U.S. Government/OU=FPKI/CN=Federal Common Policy CA G2
issuer=/C=US/O=U.S. Government/OU=FPKI/CN=Federal Common Policy CA G2
-----BEGIN CERTIFICATE-----
MIIF3TCCA8WgAwIBAgIUeW5oMyVbeJ4ygErqP3Fipiz++owDQYJKoZIhvcNAQEMBQAwXDELMakG
[snip]
sdU8kV2Mee/xJ/c0cIHZWMw0HoRZPbo=
-----END CERTIFICATE-----
subject=/C=US/O=U.S. Government/OU=FPKI/CN=Federal Common Policy CA
issuer=/C=US/O=U.S. Government/OU=FPKI/CN=Federal Common Policy CA
-----BEGIN CERTIFICATE-----
MIIEYDCCA0igAwIBAgICATAwDQYJKoZIhvcNAQELBQAwWTELMakGA1UEBhMCMCVVMx
[snip]
A2EC/0rskqTcLe4qNJMHtyznGI8=
-----END CERTIFICATE-----
subject=/C=US/O=U.S. Government/OU=NASA/OU=Certification Authorities/OU=NASA
Operational CA
issuer=/C=US/O=U.S. Government/OU=Department of the Treasury/OU=Certification
Authorities/OU=US Treasury Root CA
-----BEGIN CERTIFICATE-----
MIIF+zCCBO0gAwIBAgIETjmBFjANBgkqhkiG9w0BAQsFADCBjjELMAkGA1UEBhMC
[snip]
E+zLhSow/xs6K5uTJCuVEiaalM+HBDJ10UDjwKGMlUX1H6bI1iEnk/dYbc6Pvv8=
-----END CERTIFICATE-----
```

Note that the subject and issuer lines are included simply to make the file more human readable – all of the software tools that rely on this file will use the encoded text between the BEGIN CERTIFICATE and END CERTIFICATE lines. Actual encoded certificates will be much longer than shown above – these have been reduced for brevity.

To create an X.509 trust bundle, simply concatenate the CA certificates to be trusted (in .pem or .cer formats), adding the human readable descriptions if desired:

```
$ cat ca_cert.pem ca2_cert.cer ca3_cert.pem > ammos-ca-bundle.crt
```

Additional certs can be added after the initial creation similarly:

```
$ cat new_ca_cert.pem >> ammos-ca-bundle.crt
```

Using the default configurations, the DCS software components look for this file in /etc/pki/tls/certs/ammos-ca-bundle.crt – instructions on how to deploy this trust bundle file can be found in Section 5.2.2.4.

4.1.4.2 Java Truststore

A Java Truststore is nearly identical to the Java Keystores created earlier – the difference being that it only contains certificates, no keys. The Java utility keytool can be used to load CA certificates into a Java Truststore in JKS, PKCS#12 and BCFKS formats. JKS-formatted truststores are recommended, given the wide applicability of that format and the lack of secrets (keys) in TLS truststores. Instructions for creating each of the truststore formats and importing certificates are below. Note: each certificate imported into a Java Truststore needs a unique alias to be added to the truststore – one recommended pattern is to use a combination of the certificate Common Name (CN) and the year of expiration as the certificate alias in the truststore.

For each CA certificate, that string can be created from the output of two openssl commands:

```
$ CERT_CN=`openssl x509 -in ${TMP} -noout -text | grep -i 'subject:' | \
    awk -F',' '{print $NF}' | cut -f2 -d=' ' | cut -c1- | tr ' ' '_'`
$ CERT_EXP=`openssl x509 -in ${TMP} -noout -text | grep -i 'not after :' | \
    awk '{print $(NF-1)}'`
$ CERT_ALIAS="${CERT_CN}_${CERT_EXP}"
```

This generates aliases that look like “us_treasury_root_ca_2036” for the US Treasury Root CA certificate that expires in 2036.

4.1.4.2.1 JKS Truststore

To add a certificate to a JKS formatted truststore, or create a JKS formatted truststore, using the alias described above, run:

```
$ /bin/keytool -import -alias "${CERT_ALIAS}" -file ca_cert.pem -storetype JKS \
    -keystore ammos-truststore.jks -trustcacerts -noprompt \
    -J-Dcom.redhat.fips=false
```

The `-storepass <password>` argument can be added to the above command to avoid having to enter the truststore password each time – just take care to make sure that the command history is secured or removed once the truststore file generation is complete. The `-J-Dcom.redhat.fips=false` argument is needed on systems running with FIPS mode enabled, in order to process the cryptography needed for the older JKS format. It should have no effect on the conversion for systems with FIPS mode disabled.

4.1.4.2.2 PKCS#12 Truststore

To add a certificate to a PKCS#12 formatted truststore, or create a PKCS#12 formatted truststore, using the alias described above, run:

```
$ /bin/keytool -import -alias "${CERT_ALIAS}" -file ca_cert.pem -storetype PKCS12 \
    -keystore ammos-truststore.p12 -trustcacerts -noprompt \
    -J-Dcom.redhat.fips=false
```

The `-storepass <password>` argument can be added to the above command to avoid having to enter the truststore password each time – just take care to make sure that the command history is secured or removed once the truststore file generation is complete. The `-J-Dcom.redhat.fips=false` argument is needed on systems running with FIPS mode enabled, in order to process the cryptography needed for the PKCS#12 format. It should have no effect on the conversion for systems with FIPS mode disabled.

4.1.4.2.3 BCFKS Truststore

Generation of BCFKS keystores may require a local copy of the bc-fips Java library, if one isn't already configured for system-wide Java use. A local copy can be obtained from the Maven Central repositories at the following URI (1.0.2.5 is the latest available at the time of this release, other versions may be substituted as desired):

<https://repo.maven.apache.org/maven2/org/bouncycastle/bc-fips/1.0.2.5/bc-fips-1.0.2.5.jar>

To add a certificate to a BCFKS formatted truststore, or create a BCFKS formatted truststore, using the alias described above, run:

```
$ /bin/keytool -import -alias "${CERT_ALIAS}" -file ca_cert.pem -storetype BCFKS \
  -keystore ammos-truststore.bcfks -trustcacerts -noprompt \
  -providerpath bc-fips-1.0.2.5.jar \
  -providerclass org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider
```

The `-storepass <password>` argument can be added to the above command to avoid having to enter the truststore password each time – just take care to make sure that the command history is secured or removed once the truststore file generation is complete.

4.1.5 PREPARING MTLS TRUSTSTORES

Mutual TLS authentication also requires a list of certificates to be trusted – but a much more restrictive list than the TLS truststores generated in the previous subsection. The mTLS truststore should contain *only the specific certificates that will be allowed to access DCS Services*. For a default configuration, this should be the admin and client certificates created in Section 4.1.1.2 (`ammos-admin-cert.pem` and `ammos-client-cert.pem`). In some deployments where the Crypto Service and SDLS Service are deployed on separate systems, the mTLS truststore needs to include the System TLS Certificate from the system hosting the SDLS Service (`ammos-server-cert.pem`). The mTLS truststore is only needed in a Java Keystore format: JKS, PKCS#12, or BCFKS. BCFKS format is recommended. See the instructions in Section 4.1.4 for creating the truststore in the format desired. Name the resulting keystore `ammos-mtls-truststore.jks`, `ammos-mtls-truststore.p12`, or `ammos-mtls-truststore.bcfks` depending on the truststore format.

4.2 PREPARE CONFIGURATION INFORMATION

The DCS software requires configuration settings to be set properly. Determine the correct values for the configuration parameters before starting the software configuration process. The parameters are described in the deployment portion of this document and not repeated here. To provide a brief overview of the configuration setting preparation, the subsections below refer to the places in this document where the parameters are described.

4.2.1 PREPARING CRYPTO JAVA API AND CLI CONFIGURATION PARAMETERS

Prior to deploying the DCS Crypto Java APIs or Crypto CLIs, determine the configuration settings that are described in Section 5.2.3.1.

4.2.2 PREPARING CRYPTO SERVICE CONFIGURATION PARAMETERS

Prior to deploying the DCS Crypto Service, determine the configuration settings that are described in Section 5.2.3.2.1. If deploying the containerized Crypto Service, see the configuration settings information in Sections 5.3.3.1 and 5.3.3.2.

4.2.3 PREPARING SDLS SERVICE CONFIGURATION PARAMETERS

Prior to deploying the SDLS Service, determine the configuration settings that are described in Section 5.2.3.2.2. If deploying the containerized SDLS Service, see the configuration settings information in Sections 5.3.3.1 and 5.3.3.3.

The SDLS Service uses “managed parameters” to configure aspects of SDLS operation that are not handled by the SAs. See Section 4.2.3.1 for descriptions of the managed parameters.

4.2.3.1 Preparing Managed Parameters

Prior to deployment, the set of Managed Parameters to be used should be determined.

Managed Parameters allow the DCS SDLS software to properly calculate expected frame lengths and bit field start locations so that SDLS can be properly applied to each frame.

Each managed parameter is identified by combination of a Global Virtual Channel Identifier (GVCID) and attribute name.

A GVCID is comprised of the following parts:

- SCID: Spacecraft ID
- VCID: Virtual Channel ID
- TVFN: Transfer Frame Version Number

Note: When a segmentation header is used, the GVCID also includes the following:

- MAPID: Multiplexor Access Point ID

MAPID is not used in the managed parameters, but may be present in SAs. At this time, the managed parameters cannot be set differently for different MAPID values, but different SAs can be used for different MAPID values.

See the CCSDS 232.0-B-3 blue book for more information about each of these fields for telecommand transfer frames.

Each Security Association (SA) in the SADB must map to a set of configured Managed Parameter via the TVFN, SCID, VCID, and if segmentation headers are used, also the MAP_ID fields. For example, if an SA in the SADB has TVFN 0, SCID 44, and VCID 0, then a set of Managed Parameters for that combination of TVFN, SCID, and VCID must be configured. Also, the settings of the managed parameters on the ground side must match the managed parameters on the spacecraft.

The attribute names for TC are:

- `has_segmentation_header`: Whether the GVCID has Segmentation Headers (true/false)
- `has_pus_header`: Whether the GVICD has the Primary Utilization Standard Headers (true/false)
- `has_ecf`: Whether the GVCID has an ECF field (true/false)
- `max_frame_length`: The maximum frame length for this GVCID

The DCS SDLS Python APIs accept configuration parameters that correspond to the SDLS managed parameters. The configuration parameters are in the form:

- `'cryptolib.tc.<scid>.<vcid>.<tfvn>.<parameter>=<value>'`

The following is an example set of managed parameters that could be passed to the DCS SDLS Python API. The configuration below will set up the managed parameters for SCID 44, VCID 0, TVFN 0. (TVFN is assumed to be 0 for all managed parameters at this time).

- 'cryptolib.tc.44.0.0.has_segmentation_header=false'
- 'cryptolib.tc.44.0.0.has_pus_header=false'
- 'cryptolib.tc.44.0.0.has_ecf=true'
- 'cryptolib.tc.44.0.0.max_frame_length=1024'

Managed Parameters are configuration settings that are:

1. Specified in Command & Control application settings and passed by the application to the C or Python configuration APIs of the DCS software (i.e., for DCS C and Python APIs)
 - a. See the DCS 3.7 SIS for more information about DCS configuration parameters for command encryption -- which are passed to DCS by the command and control application (and configured there).
2. Specified in a configuration file of the SDLS Service (i.e., for the SDLS HTTPS API).

If the containerized SDLS Service will be deployed, place the set of managed parameters into a file that can be provided as an input to the container startup script via a podman (or Docker, etc.) command to create the container.

4.2.4 PREPARING SA MANAGEMENT GUI CONFIGURATION PARAMETERS

Prior to deploying the SA Management GUI, determine the configuration settings that are described in Section 5.2.3.2.3. If deploying the containerized SDLS Service, see the configuration settings information in Sections 5.3.3.1 and 5.3.3.4

4.2.5 PREPARING MARIADB SADB CONFIGURATION PARAMETERS

Prior to deploying the MariaDB SADB, determine the database configuration parameters that are described in Sections 5.4.1 through 5.4.3.

4.3 PREPARE SDLS SECURITY ASSOCIATIONS

The SDLS capabilities of DCS use Security Associations (SAs) to determine the security types (e.g., Authenticated Encryption vs. Encryption only),

4.3.1 PREPARE CSV FILE FOR POPULATING THE SADB

If DCS SDLS capabilities will be used, the project should prepare a comma separate value (CSV) file for populating the SADB. DCS contains a Security Association (SA) management capability that projects can use to manage SAs in the SADB, including the ability to add/update SAs in bulk by using a CSV file as input.

An example CSV file for creating SAs in the SADB (i.e., after the tables have been created) is contained in Appendix B. Starting with the example in Appendix C, create a CSV file that contains the entries for the SAs to be created in the SADB. Descriptions of the fields in the SA records are provided in the table below, and in some additional information after the table. The file created here in the preparation phase will be used (later, in the actual set up phase) in the instructions in Section 5.4.4.

The description, field type, and example value of each of the fields in the CSV file are described in

Table 3.

Table 3: SADB CSV Fields

Field Name	Description	Field Type	Example Value
spi	Security Parameter Index	INT	1
scid	Spacecraft ID	SMALLINT	44
vcid	Virtual Channel ID	TINYINT	1
tfvn	Transfer Frame Version Number	TINYINT	0
mapid	Multiplexor Access Point ID	TINYINT	0
sa_state	Security Association State	SMALLINT	3
st	Service Type (a.k.a., Security Type)	SMALLINT	1
ecs	Encryption Cipher Suite	VARBINARY(4)	X'01'
acs	Authentication Cipher Suite	SMALLINT	1
ekid	Encryption Key ID	VARCHAR(100) UTF8	101
akid	Authentication Key ID	VARCHAR(100) UTF8	201
abm_len	Authentication Bit Mask Length	MEDIUMINT	1024
abm	Authentication Bit Mask	VARBINARY(1024)	X'FFFFFFFF...' (for 2048 bytes)
stmacf_len	Security Trailer Message Authentication Code Field Length	SMALLINT	16
iv_len	IV length	SMALLINT	12
iv	Initialization Vector	VARBINARY(20)	X'00000000000000000000000000000001'
shivf_len	Security Header Initialization Vector Field Length	SMALLINT	12
arsn_len	Anti-Replay Counter Length	SMALLINT	0

Field Name	Description	Field Type	Example Value
arsn	Anti-Replay Counter	VARBINARY(20)	X"
shsnf_len	Security Header Sequence Number Field Length	SMALLINT	0
arsnw	Anti-Replay Counter Window	SMALLINT	5
shplf_len	Security Header Pad Length Field Length	SMALLINT	0

The SADB contains "ast" and "est" fields that are not contained in the CSV file (and are ignored if they are contained in the CSV). The "ast" and "est" fields in the SADB are set via the kmc-sa-mgmt CLI by using the "st" field. The "ast" and "est" fields are seen/used if viewing or editing the SADB contents directly (e.g. via SQL commands).

The SADB contains "acs_len" and "ecs_len" fields that are not contained in the CSV file (and are ignored if they are contained in the CSV). The "acs_len" and "ecs_len" fields in the SADB are always set to "1" by the kmc-sa-mgmt CLI. The "acs_len" and "ecs_len" are seen/used if viewing or editing the SADB contents directly (e.g. via SQL commands).

The SADB also includes an lpid field, which is not currently used and not supported by the kmc-sa-mgmt CLI.

The table below lists the SADB fields that are not read in from the CSV file.

Table 4: SADB Fields Not in CSV

Field Name	Description	Field Type	Example Value
ast	Authentication Service Type 0 - No authentication 1 - Authentication	SMALLINT	1
est	Encryption Service Type 0 - No encryption 1 - Encryption	SMALLINT	1
acs_len	Authentication Cipher Suite Length	SMALLINT	1
ecs_len	Encryption Cipher Suite Length	SMALLINT	1
lpid	Last Packet ID	SMALLINT	0

Additional information for some of the parameters in the tables above is provided below:

- **ekid:** Encryption Key ID is a reference to the cryptographic key for authenticated encryption (i.e., AES-GCM). This parameter is used in SAs for encryption and authenticated encryption, and is not used (i.e., can be null) for authentication-only SAs (which use "akid" instead).
- **ecs:** the Encryption or Authenticated Encryption algorithm. Not used for authentication-only SAs (which use "acs" instead).
 - NONE..... X'00' (default)
 - AES-GCM..... X'01'
 - AES-CBC..... X'02'
- **ecs_len:** The length of the ecs parameter. Always set to "1".
- **akid:** Authentication Key ID, used for authentication-only SAs (i.e., to identify a key for AES-CMAC, HMAC-256, or HMAC-512).
- **acs:** Authentication Cipher Suite,
 - NONE..... X'00' (default)
 - AES-CMAC..... X'01'
 - HMAC-SHA256..... X'02'
 - HMAC-SHA512..... X'03'
- **acs_len:** The length of the ecs parameter. Always set to "1".
- **sa_state:** Security Association state maps to integers as follows:
 - SA_NONE 0
 - SA_UNKEYED 1
 - SA_KEYED 2
 - SA_OPERATIONAL 3
- **tfvn/scid/vcid:** Components of the GVID that must map one-to-one to the relevant set of managed parameters for each security association!
- **st:** Security Type (a.k.a., Service Type)
 - Plaintext 0
 - Encryption 1
 - Authentication 2
 - Authenticated Encryption 3
- **est:** Encryption Service Type is set to 1 to add SDLS Encryption, or 0 to skip encryption. This field is not set directly via the kmc-sa-mgmt CLI. When using the CLI, use the "st" parameter to set est and ast.
- **ast:** Authentication Service Type is set to 1 to add SDLS Authentication, or 0 to skip authentication. This field is not set directly. Use the "st" parameter to set est and ast.
- **shivf_len:** The security header IV field length value must match the length of the provided initialization vector (for the IV) field.

- `stmacf_len`: The security trailer message authentication code field length is the length (in bytes) of the MAC field that will be added after the transfer frame data.
- `abm_len`: The authentication bit mask length must be set to the length of the supplied authentication bit mask (abm). Note that your `abm_len` must be greater than the supplied transfer frame being authenticated, the ABM is truncated. An `abm_len` of 1024, the maximum transfer frame length, is suggested.
- `abm`: The authentication bit mask to be used when creating the additional authentication data (AAD) for AES-GCM authenticated encryption, or the authentication data for the AES-CMAC algorithm for authentication-only. DCS CLI tools will assist users in creating this to their needs in the future.
- `shsnf_len`: The length of the Sequence Number (SN) field in the Security Header. Can be the same as, or less than (to transmit a truncated SN), the full length of the `arsn`.
- `arsn_len`: Length of the Sequence Number used for anti-replay. This is the full SN length, which may be greater than transmitted length (`shsnf_len`). Used for the Authentication only Security Type. The `iv_len` field is used as the length of the anti-replay counter for Authentication Encryption with AES-GCM.
- `arsn`: Anti-Replay Sequence Number. Used for Authentication only Security Type. The `iv` field is used as the anti-replay counter for Authentication Encryption with AES-GCM.
- The following fields are not currently supported:
 - `lpid`: not applicable any of the currently supported algorithms.
 - `shplf_len`: not applicable any of the currently supported algorithms.

5 DEPLOYMENT

5.1 DEPLOYMENT OVERVIEW

There are multiple methods available to deploy the DCS software components:

- Non-Containerized Deployment (DCS Client and DCS Services)
 - Direct Filesystem Deployment (see Section 5.2.1.1)
 - Archive Deployment (see Section 5.2.1.2)
 - Package Deployment (see Section 5.2.1.3)
- DCS Service Container Deployment (see Section 5.3)

Each of these methods have advantages and disadvantages – it is important to select the DCS deployment method that will simplify project use cases and long-term software maintenance. The first two methods, Direct Filesystem and Archive deployments, are best suited to development environments where there is tolerance for frequent change and differences in configurations from one system to the next. Package deployments, which deploy operating system supported packages like RPMs are best suited to stable, configuration managed environments where consistency between systems and deployments is important. Container deployments offer an easy method to deploy the DCS services as a single “black box,” but come with limitations in supported configurations.

Once a deployment method is chosen, see the preparation steps in the following subsections to prepare all necessary elements for successful DCS deployment.

5.2 NON-CONTAINERIZED DEPLOYMENT

5.2.1 INSTALL DCS SOFTWARE

As mentioned in Section 5.1, there are several deployment methods that can be used to deploy DCS for use. It is completely possible, and in many cases makes sense, to combine multiple deployment methods – using an archive file to deploy the DCS Crypto Client on some workstations and using the containerized DCS Services, for example. The subsections below contain instructions how to deploy *all* of DCS via each method, as much as each method can support.

5.2.1.1 Direct Filesystem Deployment

The Direct Filesystem Deployment method basically implies running the deployment directly from the DCS source directory after completing a build. This is done by executing the `deploy.sh` script, located in `<kmc_src_root>/kmc_resources/scripts/deploy.sh`. Using this deployment method *requires* elevated privileges to be successful, so make sure to execute the `deploy.sh` script as a privileged user, or via a privilege escalation mechanism like `sudo` (used in the examples below). This script *should* be executed from the root of the DCS Source tree. The most basic invocation of the script deploys the DCS Crypto Client into the directories configured in `<kmc_src_root>/kmc-resources/scripts/setenv.sh`, once the build is complete. See Section 3.3 for instructions on how to customize the DCS build and deployment configuration.

```
$ sudo kmc-resources/scripts/deploy.sh
```

The `deploy.sh` script supports a number of options that add additional capabilities for Direct Filesystem Deployments. The usage information for the script can be displayed with either `-h` or `--help` arguments:

```
$ kmc-resources/scripts/deploy.sh -h
Usage: kmc-resources/scripts/deploy.sh
[ -a | --all-services ] - deploy ALL KMC services
[ -c | --crypto-service ] - deploy KMC Crypto Service
[ -h | --help ] - display deploy.sh script usage
[ -i | --img ] - produce OCI container images for KMC Services
[ -m | --sa-mgmt-service ] - deploy KMC SA Management Service
[ -s | --sdls-service ] - deploy KMC SDLS REST API
[ -p DIR | --pkg DIR ] - deploy for package building in DIR
[ -r | --rpm ] - deploy and build RPM packages in kmc-resources/packaging/rpm
[ -t DIR | --tar DIR ] - deploy/build tar.bz2 archive in DIR
[ -z DIR | --zip DIR ] - deploy/build zip archive in DIR
```

In particular, the options to deploy individual services (`-c`, `-m`, `-s`, `--crypto-service`, `--sa-mgmt-service`, `--sdls-service`) or all of the services (`-a` or `--all-services`) can be applied during Direct Filesystem Deployments to additionally deploy the DCS services on the system. Deploying the services also deploys the supporting SystemD unit files and environmental configuration files in `/etc/sysconfig` for the matching service(s).

Once invoked, the `deploy.sh` script will create all of the necessary users, groups, and directories, copy over all the DCS files, and perform some of the most basic configuration settings based on the `CRYPTO_SERVICE_FQDN`, `SADB_FQDN`, and various `PATH` variables set in `<kmc_src_root>/kmc-resources/scripts/setenv.sh` prior to the DCS build.

5.2.1.2 Archive Deployment

Archive Deployments are affected by using the `deploy.sh` script to create an archive file – either a tar/bzip2 or zip archive – that contains a DCS deployment. To create an archive file after completing a DCS build, use the `deploy.sh` script as follows, substituting the packaging directory as appropriate. Building an archive file *requires* elevated privileges to include important ownership and permission settings; creating an archive file *without* elevated privileges will result in the current user owning all of the files and directories in the archive. By default, the `deploy.sh` script will not include the DCS services in archive files – make sure to include the service enabling flags (`-c`, `-m`, `-s`, `--crypto-service`, `--sa-mgmt-service`, `--sdls-service`, `-a`, or `--all-services`) to include desired services in the archive file.

```
$ kmc-resources/scripts/deploy.sh --tar /tmp/kmc
```

The `deploy.sh` script will, invoked as above, will perform the following steps:

1. Create a `/tmp/kmc` directory. If `/tmp/kmc` already exists, it will be deleted and recreated.
2. Deploy the DCS Crypto Client inside of that directory
3. Create a tar/bzip2 archive (`kmc-3.7.0.tar.bz2`, based on the `VERSION` variable in `<kmc_src_root>/kmc-resources/scripts/setenv.sh`) with all the files in the deployment
4. Remove all files in `/tmp/kmc` *other* than the resulting archive file.

This will leave just the archive file in `/tmp/kmc/kmc-3.7.0.tar.bz2`, which can be used on desired systems to deploy the DCS Crypto Client.

Important Note: Archive Deployments can only deliver directories and files, and cannot create users or groups or execute configuration changes. Take care to make sure that all the users and groups defined in `<kmc_src_root>/kmc-resources/scripts/setenv.sh` are defined on systems prior to performing an Archive Deployment.

Once the archive file has been successfully created, it can be used on desired systems – just expand the archive like any similarly-formatted file, from the root directory as a privileged user:

```
$ sudo tar -xjf /tmp/kmc-3.7.0.tar.bz2
```

That will create all the directories and install all the files included in the archive file on the local system.

5.2.1.3 RPM Package Deployment

Package Deployments are largely considered standard in most Linux environments. DCS currently supports the production of Red Hat-based RPM packages for DCS deployments. These packages include all the directories and files for the DCS deployments, can create and manage system level resources like users and groups, ensure that software dependencies are met, and generally make it easier to manage upgrades, downgrades, and other package level changes.

The DCS package structure includes four packages. The first is the base DCS package, `kmc-3.7.0-1.el9.x86_64.rpm`, includes the DCS Crypto Client. The other three packages, `kmc-crypto-service-3.7.0-1.el9.x86_64.rpm`, `kmc-sa-mgmt-service-3.7.0-1.el9.x86_64.rpm`, and `kmc-sdls-service-3.7.0-1.el9.x86_64.rpm`, each contain one of the DCS Services, to facilitate distributed deployments. Please note that the DCS Services package all require the installation of the main DCS package in order to meet software dependencies.

The DCS RPM packages are produced with the `deploy.sh` script, after a successful DCS build. From the DCS Source root, the following command will produce all four DCS packages. This step *does not* require elevated privileges, and should always be run as the normal (non-privileged) user that performed the DCS build.

```
$ kmc-resources/scripts/deploy.sh --rpm
```

The command above will install the full DCS deployment, including all of the DCS services, into a local subdirectory in the DCS Source tree -- `<kmc_src_root>/kmc-resources/packaging/rpm/BUILD/kmc-3.7.0` (based on the `VERSION` variable in `kmc-resources/scripts/setenv.sh`), and then use the RPM Specification in `<kmc_src_root>/kmc-resources/packaging/rpm/SPECS/kmc.spec` to build the DCS RPM packages. The RPM packages produced end up in `<kmc_src_root>/kmc-resources/packaging/rpm/RPMS/x86_64`.

These RPM packages can and should be deployed with standard system tools. The ideal situation is to upload the DCS RPM packages to a YUM repository that allows network access from local systems – that will enable all of the intended systems to duplicate deployment of desired DCS software components. Deploying the packages from a YUM repository also facilitates software dependency resolution – if there are unmet software dependencies, `dnf`, the system tool designed to interact with YUM repositories, will attempt to resolve them by adding additional software packages where available.

To deploy all the DCS RPM packages with `dnf`, once the target system has been configured with an appropriate YUM repository, use:

```
$ sudo dnf install kmc-3.7.0 kmc-crypto-service-3.7.0 kmc-sdls-service-3.7.0 kmc-sa-
mgmt-service-3.7.0
```

The DCS packages can also be deployed without a YUM Repository, either with the `dnf localinstall` option, or directly with the `rpm` tool. Examples of both are included below:

```
$ sudo dnf localinstall kmc-3.7.0-1.el9.x86_64.rpm
$ sudo rpm -ivH kmc-3.7.0-1.el9.x86_64.rpm
```

All package installations using RPM packages *require* elevated privileges. The examples above all use `sudo` for that, but they could also be executed directly by the root user, or another privileged system user.

5.2.2 INSTALL CRYPTOGRAPHIC ARTIFACTS

All the cryptographic artifacts generated in Section 4.1 need to be installed to support DCS deployment. For each of the artifacts generated, instructions are provided that include which directory to install the artifact in, the filename to use for each artifact, owner, group and permission settings for the artifact that will allow all of the necessary users and services to interact properly with it. It is acceptable to change filenames and installation locations for these artifacts as needed, but doing so will likely require significant configuration changes for the DCS software components.

The cryptographic artifacts can be installed via many methods – direct copy, via OS packages or archives, through a configuration management system like Puppet or Ansible, and probably many others. The instructions provided in the subsections below provide the locations, filenames, owner, group and permission settings for each of the artifacts, and simple filesystem copy instructions.

5.2.2.1 Install TLS Keys & Certificates

On RHEL systems, and RHEL-derived systems like Oracle Linux, AlmaLinux and Rocky Linux, the standard system installation location for TLS certificates is `/etc/pki/tls/certs`, and TLS keys and other sensitive artifacts get installed into `/etc/pki/tls/private`. The DCS software components are configured to look in these locations by default.

TLS certificates are usually not considered sensitive, as they require the matching TLS private key to enable use. At a minimum, the `ammos-server-cert.pem` generated in Section 4.1.1.1 for the system hostname where the DCS software will be hosted must be installed.

If DCS is being configured to use mTLS authentication, the `ammos-admin-cert.pem` and `ammos-client-cert.pem` generated in Section 4.1.1.2 also need be deployed into this directory, along with any other TLS certificates generated for mTLS authentication, but they require different owner, group and permission settings.

`ammos-admin-cert.pem`:

- Location: `/etc/pki/tls/certs`
- Owner: root
- Group: crypto
- Permissions: 0440 (owner read, group read, world no access)

amos-client-cert.pem:

- Location: /etc/pki/tls/certs
- Owner: root
- Group: users
- Permissions: 0440 (owner read, group read, world no access)

Other certificates intended for use in mTLS authentication should be installed with similar permissions, where filesystem access is controlled by a POSIX group limited to users that need to use the certificate to interact with the DCS services.

TLS private keys *are* considered sensitive files – access to the private key file, and passphrase if any is set, enable use of the matching certificate. At a minimum, the amos-server-key.pem generated in Section 4.1.1.1 for the system hostname where the DCS software will be hosted must be installed as follows:

amos-server-key.pem:

- Location: /etc/pki/tls/private
- Owner: root
- Group: crypto (see note below)
- Permissions: 0440 (owner read, group read, world no access)

Note: many system services and applications can use the TLS System keypair to provide TLS-encrypted connections. If additional services are being configured on the host where DCS is being deployed, it may be advantageous to use a different group for the key above – a POSIX group that contains the crypto user, the MariaDB user (if the host is also running MariaDB for the SADB component, for example), the httpd user (if httpd is being configured), etc. This group is named amos-tls in prior versions of this guide, but it can be named anything convenient as long as it contains all the service execution users that will need access.

If DCS is being deployed to use mTLS authentication, the amos-admin-key.pem and amos-client-key.pem generated in Section 4.1.1.2 must also be deployed into this directory, along with any other TLS keys generated for mTLS authentication, but they require different owner, group and permission settings.

amos-admin-key.pem:

- Location: /etc/pki/tls/private
- Owner: root
- Group: crypto
- Permissions: 0440 (owner read, group read, world no access)

amos-client-key.pem:

- Location: /etc/pki/tls/private
- Owner: root
- Group: users
- Permissions: 0440 (owner read, group read, world no access)

Other keys intended for use in mTLS authentication should be installed with similar permissions, where filesystem access is controlled by a POSIX group limited to users that need to use the certificate to interact with the DCS services.

To install all the artifacts listed above, run the following commands with elevated privileges (as root, via sudo, etc.):

```
$ /bin/cp ammos-server-cert.pem /etc/pki/tls/certs/
$ /bin/chown root:root /etc/pki/tls/certs/amos-server-cert.pem
$ /bin/chmod 0444 /etc/pki/tls/certs/amos-server-cert.pem
$ /bin/cp ammos-server-key.pem /etc/pki/tls/private/
```

If using a group other than crypto for the TLS Server key, substitute that group name in the next command:

```
$ /bin/chown root:crypto /etc/pki/tls/private/amos-server-key.pem
$ /bin/chmod 0440 /etc/pki/tls/private/amos-server-key.pem
```

If configuring DCS for mTLS authentication, the following commands can be run to install the generated artifacts:

```
$ /bin/cp ammos-admin-cert.pem ammos-client-cert.pem /etc/pki/tls/certs/
$ /bin/chown root:crypto /etc/pki/tls/certs/amos-admin-cert.pem
$ /bin/chown root:users /etc/pki/tls/certs/amos-client-cert.pem
$ /bin/chmod 0440 /etc/pki/tls/certs/amos-[admin|client]-cert.pem
$ /bin/cp ammos-admin-key.pem ammos-client-key.pem /etc/pki/tls/private/
$ /bin/chown root:crypto /etc/pki/tls/private/amos-admin-key.pem
$ /bin/chown root:users /etc/pki/tls/private/amos-client-key.pem
$ /bin/chown 0440 /etc/pki/tls/private/amos-client-key.pem
$ /bin/chown 0440 /etc/pki/tls/private/amos-admin-key.pem
```

5.2.2.2 Install TLS Keystores

TLS keystore files are also considered sensitive files – they contain both TLS keys and certificates. At a minimum, DCS deployments require at least one form of the TLS System keystore generated in Section 4.1.2 – JKS, PKCS#12, or BCFKS. Use of the BCFKS format is recommended for all keystores that will be used with DCS. The instructions below are for BCFKS formatted keystores – to use a different format, just replace the file suffix with the correct one for the specific file format: JKS keystores use .jks, PKCS#12 keystores use .p12, and BCFKS keystores use .bcfks.

amos-system-keystore.bcfks:

- Location: /etc/pki/tls/private
- Owner: root
- Group: crypto (see note below)
- Permissions: 0440 (owner read, group read, world no access)

Note: many system services and applications can use the TLS System keystore to provide TLS-encrypted connections. If additional services are being configured on the host where DCS is being deployed, it may be advantageous to use a different group for the keystore above – a POSIX group that contains the crypto user, the MariaDB user (if the host is also running MariaDB for the SADB component, for example), the httpd user (if httpd is being configured), etc. This group is named ammos-tls in prior versions of this guide, but it can be named anything convenient as long as it contains all the service execution users that will need access.

If DCS is being deployed to use mTLS authentication, the `ammos-admin-keystore.bcfks` and `ammos-client-keystore.bcfks` generated in Section 4.1.2.2 must also be deployed into this directory, along with any other TLS keystores generated for mTLS authentication, but they require different owner, group and permission settings.

`ammos-admin-keystore.bcfks`:

- Location: `/etc/pki/tls/private`
- Owner: `root`
- Group: `crypto`
- Permissions: `0440` (owner read, group read, world no access)

`ammos-client-keystore.bcfks`:

- Location: `/etc/pki/tls/private`
- Owner: `root`
- Group: `users`
- Permissions: `0440` (owner read, group read, world no access)

To install all the artifacts listed above, run the following commands with elevated privileges (as root, via sudo, etc.):

```
$ /bin/cp ammos-server-keystore.bcfks /etc/pki/tls/private/
```

If using a group other than `crypto` for the TLS Server keystore, substitute that group name in the next command:

```
$ /bin/chown root:crypto /etc/pki/tls/private/ammos-server-keystore.bcfks
$ /bin/chmod 0440 /etc/pki/tls/private/ammos-server-keystore.bcfks
```

If configuring DCS for mTLS authentication, the following commands can be run to install the generated artifacts:

```
$ /bin/cp ammos-admin-keystore.bcfks ammos-client-keystore.bcfks /etc/pki/tls/private/
$ /bin/chown root:crypto /etc/pki/tls/private/ammos-admin-keystore.bcfks
$ /bin/chown root:users /etc/pki/tls/private/ammos-client-keystore.bcfks
$ /bin/chown 0440 /etc/pki/tls/private/ammos-[admin|client]-keystore.bcfks
```

5.2.2.3 Install Crypto Keystore

The Crypto Keystore contains the cryptographic keys that will be used by the DCS Crypto Service to perform cryptographic operations. The Crypto Service can be configured to use a Crypto Keystore, a KMIP-compliant Key Management Service (KMS), or both. As such, it should be considered very sensitive, and protected strictly so that only the Crypto Service execution user can access it.

The Crypto Keystore is usually installed alongside the DCS Crypto Service, and configured so that it is only readable by the service user executing the DCS Crypto Service. To install the Crypto Keystore generated in Section 4.1.3, execute the following steps. The examples below are for a BCFKS-formatted keystore, but keystores written in other formats (JKS, PKCS#12) can be installed using the same commands by replacing the file suffix – JKS uses `.jks`, PKCS#12 uses `.p12`. The instructions below also use the default installation paths for DCS and the DCS Crypto Service. If the installation paths for the DCS Crypto Service have been customized, as described in Section 3.3, adjust the installation locations in the following commands to match.

```
$ /bin/cp crypto-keystore.bcfks /opt/amos/kmc/services/crypto-service/etc/
$ /bin/chown root:crypto \
/opt/amos/kmc/services/crypto-service/etc/crypto-keystore.bcfks
$ /bin/chmod 0440 /opt/amos/kmc/services/crypto-service/etc/crypto-keystore.bcfks
```

5.2.2.4 Install TLS Truststores

TLS Truststores contain the CA Certificates that form the basis for trust chains for TLS Certificates. As the truststores contain the certificates that the Certificate Authorities usually publish publicly, the TLS truststore files are not considered sensitive. DCS software components are configured to look for the TLS truststores in two different files, based on the format required for each component. The process for generating these files was described in Section 4.1.4, but many sites and projects have curated truststores available for use, and most operating systems have a system-level set that can often be used if nothing more specific is available. Instructions for configuring all three scenarios are provided below.

5.2.2.4.1 Installing Generated Truststores

The truststore(s) generated in Section 4.1.4 can be installed for DCS with the following commands. Substitute the correct file extension for the Java Keystore format in use, if not BCFKS.

```
$ /bin/cp ammos-ca-bundle.crt ammos-truststore.bcfks /etc/pki/tls/certs/
$ /bin/chown root:root /etc/pki/tls/certs/amos-ca-bundle.crt \
/etc/pki/tls/certs/amos-truststore.bcfks
$ /bin/chmod 0444 /etc/pki/tls/certs/amos-ca-bundle.crt \
/etc/pki/tls/certs/amos-truststore.bcfks
```

5.2.2.4.2 Installing Site/Project Truststores

Obtain the site or project truststore from the distribution source. DCS needs two different formats to support the various components: an X.509 Trust bundle (text file containing the CA certificates to be trusted) and a Java Truststore in one of the Java Keystore formats (JKS, PKCS#12, or BCFKS). JKS is recommended.

The site or project truststores can be installed directly as the truststore files that DCS is configured to use, or can be installed as per standard site/project practices and symbolically linked to the locations configured for DCS – both approaches will work.

If installing the files directly, install the X.509 Trust Bundle to `/etc/pki/tls/certs/amos-ca-bundle.crt`, and the Java Truststore to `/etc/pki/tls/certs/amos-truststore.jks` (substitute the correct file extension for the Java Keystore format in use, if not JKS). Both files should have owner root, group root, and permissions set to 0444 (owner read, group read, world read).

```
$ /bin/cp proj_trust_bundle.crt /etc/pki/tls/certs/amos-ca-bundle.crt
$ /bin/cp proj_truststore.bcfks /etc/pki/tls/certs/amos-truststore.bcfks
$ /bin/chown root:root /etc/pki/tls/certs/amos-ca-bundle.crt \
/etc/pki/tls/certs/amos-truststore.bcfks
$ /bin/chmod 0444 /etc/pki/tls/certs/amos-ca-bundle.crt \
/etc/pki/tls/certs/amos-truststore.bcfks
```

To use symlinks to site or project truststores installed in other locations, adjust the following commands to reference the correct locations for the truststore files:

```
$ /bin/ln -s /some/other/location/proj_trust_bundle.crt \
/etc/pki/tls/certs/amos-ca-bundle.crt
```

```
$ /bin/ln -s /some/other/location/proj_truststore.jks \
/etc/pki/tls/certs/amos-truststore.jks
```

5.2.2.4.3 Using the System Truststores

Using the operating system provided truststores should generally be considered a fall back – good security practice is to limit the trust anchors to necessary Certificate Authorities. However, the operating system provided truststores normally include a wide range of commercial and governmental Certificate Authorities. The truststores provided by the operating system vary widely in filesystem location, from one system provider to another. The instructions below are suitable for Red Hat Enterprise Linux, and many of the RHEL-derived Linux variants such as Oracle Linux, AlmaLinux, and Rocky Linux.

```
$ /bin/ln -s /etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem \
/etc/pki/tls/certs/amos-ca-bundle.crt
$ /bin/ln -s /etc/pki/ca-trust/extracted/java/cacerts \
/etc/pki/tls/certs/amos-truststore.jks
```

5.2.2.5 Install mTLS Truststore

If DCS is being deployed to use mTLS authentication, the mTLS Truststore generated in Section 4.1.5 must also be deployed. This file contains only certificates, but is considered sensitive since it controls access to the DCS Services. The instructions below are for a JKS-formatted truststore – substitute the correct suffix in the filename if the mTLS truststore uses a different format.

amos-mtls-truststore.jks:

- Location: /etc/pki/tls/private
- Owner: root
- Group: crypto
- Permissions: 0440 (owner read, group read, world no access)

To install the mTLS truststore generated earlier, use the following commands, substituting the filename prefix as needed for alternate truststore formats:

```
$ /bin/cp amos-mtls-truststore.jks /etc/pki/tls/private/
$ /bin/chown root:crypto /etc/pki/tls/private/amos-mtls-truststore.jks
$ /bin/chmod 0440 /etc/pki/tls/private/amos-mtls-truststore.jks
```

5.2.3 CONFIGURE NON-CONTAINERIZED DCS SOFTWARE

5.2.3.1 Configure the DCS Crypto Client

After installing the DCS Crypto Client software, configure the Crypto Java APIs (which is not necessary for command encryption, but the later steps for editing kmc-sa-mgmt.properties and setting up access to log files are necessary for command encryption) by editing the following file:

/opt/amos/kmc/etc/kmc-crypto.cfg

This configuration file specifies which cryptography providers to use for the various algorithms and which key/cert/keystore to use for mTLS to the Crypto Service and/or KMS. The file also specifies if a KMS and/or Crypto Keystore will be used (and parameters associated with the KMS or keystore). The DCS Crypto Client *defaults* to using a keystore, if one is configured, even if a KMS is *also* configured. If the key is not found in the Crypto Keystore (if specified), the KMS (if specified) will be used to retrieve the key.

Below is an example of the kmc-crypto.cfg file:

```
#
# Configuration file for KMC Crypto Client
#

# Specify a KMS and/or Crypto Keystore to use as source(s) of keys.

# Keys are requested from the specified KMS if not found in the specified keystore.
#key_management_service_uri=tls12://fully-qualified-domain-name:5696

# crypto_keystore_type can be BCFKS, PKCS12, JCEKS, or JKS. FIPS should use BCFKS keystore.
# crypto_key_password is not used by PKCS12 keystore
crypto_keystore_location=/opt/ammos/kmc/etc/crypto-keystore.bcfks
crypto_keystore_type=BCFKS
crypto_keystore_password=m4kAg0odPw:)
#crypto_key_password=PASSWORD_OF_ALL_KEYS

# Crypto Service URL - used by Crypto CLIs:
crypto_service_uri=https://crypto-srv1.example.com:8443/crypto-service

# authentication to Crypto Service: either SSO or mTLS
#crypto_service_sso_cookie_file=$HOME/.cam_cookie_file
#crypto_service_mtls_client_type=PEM
crypto_service_mtls_client_key=/etc/pki/tls/private/ammos-client-key.pem
crypto_service_mtls_client_cert=/etc/pki/tls/certs/ammos-client-cert.pem

# keystore for authenticating with KMS when TLS is used (FIPS should use .bcfks)
#keystore_file=/etc/pki/tls/private/ammos-server-keystore.bcfks
#keystore_file=/etc/pki/tls/private/ammos-server-keystore.p12
#keystore_password=changeit

# parameters for connection to KMS
# default cipher suites: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384, TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
# additional_cipher_suites="TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384, TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384"
connection_timeout_connect=5000
connection_timeout_read=30000

# key cache (time-to-live TTL in minutes)
key_cache_enable=true
key_cache_ttl=1440

# Crypto algorithms should use Java standard names:
# http://docs.oracle.com/javase/7/docs/technotes/guides/security/StandardNames.html
default_secure_random_algorithm=SHA1PRNG
allowed_secure_random_algorithms=SHA1PRNG

default_symmetric_encryption_algorithm=AES-256
allowed_symmetric_encryption_algorithms=AES-128:AES-256:DESede-168
default_asymmetric_encryption_algorithm=RSA-2048
allowed_asymmetric_encryption_algorithms=RSA-2048:RSA-3072:RSA-4096

default_message_digest_algorithm=SHA-256
allowed_message_digest_algorithms=SHA-1:SHA-256:SHA-384:SHA-512
default_hmac_algorithm=HmacSHA256
allowed_hmac_algorithms=HmacSHA1:HmacSHA256:HmacSHA384:HmacSHA512
default_cmac_algorithm=AESCMAC
allowed_cmac_algorithms=AESCMAC:DESedeCMAC
default_digital_signature_algorithm=SHA256withRSA
allowed_digital_signature_algorithms=SHA1withRSA:SHA256withRSA:SHA384withRSA:SHA512withRSA

default_AES_transformation=AES/CBC/PKCS5Padding
allowed_AES_transformations=AES/CBC/PKCS5Padding:AES/GCM/NoPadding
default_DESede_transformation=DESede/CBC/PKCS5Padding
allowed_DESede_transformations=DESede/CBC/PKCS5Padding
default_RSA_transformation=RSA/ECB/OAEPWithSHA-256AndMGF1Padding
allowed_RSA_transformations=RSA/ECB/PKCS1Padding:RSA/ECB/OAEPWithSHA-256AndMGF1Padding:RSA/ECB/OAEPWithSHA1AndMGF1Padding
```

```
# Default Providers
# https://docs.oracle.com/javase/8/docs/technotes/guides/security/SunProviders.html
# Uncomment and change value if non-default provider is used.
#SHA1PRNG_provider=SUN
#AES_provider=SunJCE
#DESede_provider=SunJCE
#RSA_provider=SunJCE
#SHA-1_provider=SUN
#SHA-256_provider=SUN
#SHA-384_provider=SUN
#SHA-512_provider=SUN
#HmacSHA1_provider=SunJCE
#HmacSHA256_provider=SunJCE
#HmacSHA384_provider=SunJCE
#HmacSHA512_provider=SunJCE
#SHA1withRSA_provider=SunRsaSign
#SHA256withRSA_provider=SunRsaSign
#SHA384withRSA_provider=SunRsaSign
#SHA512withRSA_provider=SunRsaSign

# Provider classes
#SUN=sun.security.provider.Sun
#SunJCE=com.sun.crypto.provider.SunJCE
BCFIPS_provider_class=org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider

# Bouncy Castle FIPS provider
AES_provider=BCFIPS
AESCMAC_provider=BCFIPS
DESedeCMAC_provider=BCFIPS
AESGCM_provider=BCFIPS
```

To use the `/opt/ammos/kmc/bin/kmc-sa-mgmt` command line interface (CLI), the following file must be configured:

- `/opt/ammos/kmc/etc/kmc-sa-mgmt.properties`

An example of the `kmc-sa-mgmt.properties` file is shown below.

```
# database user name
db.auth.user=adminuser
# database password
#db.auth.pass=none
# database connection string. If provided, the db.host, db.port, and db.schema keys will be ignored
db.conn.string=none
# database host
db.host=sadb.example.com
# database port
db.port=3306
# database schema
db.schema=sadb
# database TLS keystore
db.keystore=/etc/pki/tls/private/ammos-admin-keystore.p12
# database TLS keystore password
db.keystore.pass=k3ySt0Rp4ss
# database TLS truststore
db.truststore=/etc/pki/tls/certs/ammos-truststore.jks
# database TLS truststore password
db.truststore.pass=changeit
# use TLS for database connections
# the provided truststore must contain the certificate chain for connecting to the database.
Db.tls=true
# use mutual TLS for database connections. When mTLS is true, the db.auth.pass key is ignored.
# the provided keystore must be trusted by the database server and database user.
Db.mtls=true
```

As shown in the example above, using mTLS to the SADB is recommended. To use mTLS to the SADB: Set `db.auth.user` to the username in the SADB who is allowed to manage SAs, set `db.host` to the fully-qualified domain name (FQDN) of the host where the SADB is located, set `db.keystore` to the admin user client keystore location, set `db.keystore.pass` to the password set in Section 4.1.2, and set `db.tls` to true. Do not set `db.auth.pass` when using mTLS to the SADB.

The logging configuration parameters are contained in the following files:

- `kmc-crypto-log4j.properties` - for data-level cryptography API logging
- `kmc-sa-mgmt-log4j2.xml` - for `kmc-sa-mgmt` CLI logging

The Crypto Client writes its log files under `/opt/ammos/kmc/logs/`, which must be readable & writable by all users who use client-side applications that use the Crypto Client. The default permissions make `/opt/ammos/kmc/logs/` (and log files in it) writable by the "users" group. Either put the users into the "users" group, or change the group owner of the `/opt/ammos/kmc/logs/` directory, and files in it, to a group that will contain the users (e.g., a "cmdenc" group). For example:

```
$ sudo chgrp -R cmdenc /opt/ammos/kmc/logs
```

There are no configuration files for command encryption in the DCS Crypto Client. Configuration of command encryption is done via the SDLs APIs (see the DCS 3.7 SIS). The command and control application may have its own configuration parameters for command encryption capabilities in DCS. See the installation instructions for the command and control application for information about the relevant configuration parameters for the application.

5.2.3.2 Configure the DCS Services

5.2.3.2.1 Configuring the Crypto Service

To use the DCS Crypto Service (on its own or with command encryption), it needs to be configured so that it knows the location of the Key Management Service (KMS) and/or Crypto Keystore, and parameters for Transport Layer Security (TLS), etc. Those parameters are set in the following DCS config file:

- `/opt/ammos/kmc/services/crypto-service/etc/kmc-crypto.cfg`

When using a Crypto Keystore, set the following (otherwise, comment them out) in `kmc-crypto.cfg`:

- `crypto_keystore_location=<full path to Crypto Keystore>`
 - For example: `crypto_keystore_location = /opt/ammos/kmc/etc/crypto-keys.bcfks`
- `crypto_keystore_type=<BCFKS, PKCS12, or JKS>`
 - For example: `crypto_keystore_type=BCFKS`
- `crypto_keystore_password=<keystore password>`
 - For example: `crypto_keystore_password=y0urK3strP@s$wd`

When using a KMS, set the following parameters (else, comment them out) in `kmc-crypto.cfg`:

- `key_management_service_uri=tls12://fully-qualified-domain-name:5696`
 - where 5696 is the default KMIP port number for TLS1.2. If the KMS uses a different port number, it should be changed accordingly in the `kmc-crypto.cfg` file
- `keystore_file=/etc/pki/tls/private/amos-server-keystore.bcfks`
- `keystore_password=changeit`
 - Can use this default, if it was used when creating the keystore.

The last two parameters above are used for mutual TLS authentication with the KMS.

The following is an example of the `kmc-crypto.cfg` file for the DCS Crypto Service:

```
#
# Configuration file for KMC Crypto Service
#

# Specify a KMS and/or Crypto Keystore to use as source(s) of keys.

# Keys are requested from the specified KMS if not found in the specified keystore.
#key_management_service_uri=tls12://fully-qualified-domain-name:5696

# crypto_keystore_type can be BCFKS, PKCS12, JCEKS, or JKS. FIPS should use BCFKS keystore.
# crypto_key_password is not used by PKCS12 keystore.
crypto_keystore_location=/opt/amos/kmc/etc/project-keys.bcfks
crypto_keystore_type=BCFKS
crypto_keystore_password=k3Yst0rep3s$
#crypto_key_password=PASSWORD_OF_ALL_KEYS

# keystore for authenticating with KMS when TLS is used (FIPS operation
# requires .bcfks)
keystore_file=/etc/pki/tls/private/amos-server-keystore.bcfks
#keystore_file=/etc/pki/tls/private/amos-server-keystore.pl2
keystore_password=changeit

# parameters for connection to KMS
# default cipher suites: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,
# TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
#additional_cipher_suites="TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
#TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384"
connection_timeout_connect=5000
connection_timeout_read=30000

# key cache (time-to-live TTL in minutes)
key_cache_enable=true
key_cache_ttl=1440

# Crypto algorithms should use Java standard names:
# https://docs.oracle.com/en/java/javase/17/docs/specs/security/standard-names.html

default_symmetric_encryption_algorithm=AES-256
allowed_symmetric_encryption_algorithms=AES-128:AES-256:DESede-168
default_asymmetric_encryption_algorithm=RSA-2048
allowed_asymmetric_encryption_algorithms=RSA-2048:RSA-3072:RSA-4096

default_message_digest_algorithm=SHA-256
allowed_message_digest_algorithms=SHA-1:SHA-256:SHA-384:SHA-512
default_hmac_algorithm=HmacSHA256
allowed_hmac_algorithms=HmacSHA1:HmacSHA256:HmacSHA384:HmacSHA512
default_cmac_algorithm=AESCMAC
allowed_cmac_algorithms=AESCMAC:DESedeCMAC
default_digital_signature_algorithm=SHA256withRSA
allowed_digital_signature_algorithms=SHA1withRSA:SHA256withRSA:SHA384withRSA:SHA512withRSA

default_AES_transformation=AES/CBC/PKCS5Padding
allowed_AES_transformations=AES/CBC/PKCS5Padding:AES/GCM/NoPadding
```

```

default_DESede_transformation=DESede/CBC/PKCS5Padding
allowed_DESede_transformations=DESede/CBC/PKCS5Padding
default_RSA_transformation=RSA/ECB/OAEPWithSHA-256AndMGF1Padding
allowed_RSA_transformations=RSA/ECB/PKCS1Padding:RSA/ECB/OAEPWithSHA-
256AndMGF1Padding:RSA/ECB/OAEPWithSHA1AndMGF1Padding

# Default Providers
# https://docs.oracle.com/en/java/javase/17/security/oracle-providers.html
# Uncomment and change value if non-default provider is used.
#AES_provider=SunJCE
#DESede_provider=SunJCE
#RSA_provider=SunJCE
#SHA-1_provider=SUN
#SHA-256_provider=SUN
#SHA-384_provider=SUN
#SHA-512_provider=SUN
#HmacSHA1_provider=SunJCE
#HmacSHA256_provider=SunJCE
#HmacSHA384_provider=SunJCE
#HmacSHA512_provider=SunJCE
#SHA1withRSA_provider=SunRsaSign
#SHA256withRSA_provider=SunRsaSign
#SHA384withRSA_provider=SunRsaSign
#SHA512withRSA_provider=SunRsaSign

# Provider classes
#SUN=sun.security.provider.Sun
#SunJCE=com.sun.crypto.provider.SunJCE
BCFIPS_provider_class=org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider

# Bouncy Castle FIPS provider
AES_provider=BCFIPS
AESCMAC_provider=BCFIPS
DESedeCMAC_provider=BCFIPS
AESGCM_provider=BCFIPS

```

5.2.3.2.2 Configuring the SDLS Service

To use the SDLS Service, the `/opt/ammos/kmc/services/sdls-service/etc/kmc-sdls-service.properties` configuration file must be edited to add the correct parameter values.

An example of the `kmc-sdls-service.properties` is shown below. In this example, mTLS is used to prevent unauthorized access to the service.

```

# KMC SDLS Service Properties File
#
# See the KMC Product Guide or KMC Software Interface Specification (SIS)
# for descriptions of each supported property.
#
#CryptoLib MariaDB SADB Properties
cryptolib.sadb.type=mariadb
cryptolib.sadb.mariadb.fqdn=sadb123.example.com
cryptolib.sadb.mariadb.fqdn=sadb.example.com
cryptolib.sadb.mariadb.username=cryptosvc
#Password not necessary when using mTLS
cryptolib.sadb.mariadb.password=
cryptolib.sadb.mariadb.mtls.clientkey=/etc/pki/tls/private/ammos-server-key.pem
cryptolib.sadb.mariadb.mtls.clientcert=/etc/pki/tls/certs/ammos-server-cert.pem
cryptolib.sadb.mariadb.require_secure_transport=true
cryptolib.sadb.mariadb.tls.verifyserver=true
cryptolib.sadb.mariadb.tls.cacert=/etc/pki/tls/certs/ammos-ca-bundle.crt
#CryptoLib KMC Crypto Service Properties

```



```

cryptolib.crypto.type=kmccryptoservice
cryptolib.crypto.kmccryptoservice.protocol=https
#cryptolib.crypto.kmccryptoservice.fqdn=cryptol23.example.com
cryptolib.crypto.kmccryptoservice.fqdn=crypto.example.com
cryptolib.crypto.kmccryptoservice.app=crypto-service

cryptolib.crypto.kmccryptoservice.mtls.clientkey=/etc/pki/tls/private/amos-server-key.pem
cryptolib.crypto.kmccryptoservice.mtls.clientcert=/etc/pki/tls/certs/amos-server-cert.pem
cryptolib.crypto.kmccryptoservice.mtls.clientcertformat=PEM
cryptolib.crypto.kmccryptoservice.verifyserver=true
cryptolib.crypto.kmccryptoservice.cacert=/etc/pki/tls/certs/amos-ca-bundle.crt

#cryptolib.cam.enabled=true
#cryptolib.cam.login_method=KEYTAB_FILE
#cryptolib.cam.cookie_file=
#cryptolib.cam.keytab_file=
#cryptolib.cam.access_manager_uri=
#cryptolib.cam.username=
#cryptolib.cam.cam_home=/amos/css

#CryptoLib Global Behavioral Properties
cryptolib.tc.vcid_bitmask=0x3f
cryptolib.apply_tc.create_ecf=true
cryptolib.process_tc.check_fecf=false
cryptolib.process_tc.ignore_antireplay=true
cryptolib.process_tc.ignore_sa_state=true
cryptolib.process_tc.process_pdus=false

#Managed Parameters
# Managed Parameters used for DCS testing
cryptolib.tc.44.0.0.has_segmentation_header=false
cryptolib.tc.44.0.0.has_pus_header=false
cryptolib.tc.44.0.0.has_ecf=true
cryptolib.tc.44.0.0.max_frame_length=1024
cryptolib.tc.44.1.0.has_segmentation_header=false
cryptolib.tc.44.1.0.has_pus_header=false
cryptolib.tc.44.1.0.has_ecf=true
cryptolib.tc.44.1.0.max_frame_length=1024
cryptolib.tc.44.2.0.has_segmentation_header=false
cryptolib.tc.44.2.0.has_pus_header=false
cryptolib.tc.44.2.0.has_ecf=true
cryptolib.tc.44.2.0.max_frame_length=1024
cryptolib.tc.44.3.0.has_segmentation_header=false
cryptolib.tc.44.3.0.has_pus_header=false
cryptolib.tc.44.3.0.has_ecf=true
cryptolib.tc.44.3.0.max_frame_length=1024

# Example managed parameters
#cryptolib.tc.<scid>.<vcid>.<tfvn>.<property>
#cryptolib.tc.44.0.0.has_segmentation_header=false
#cryptolib.tc.44.0.0.has_pus_header=false
#cryptolib.tc.44.0.0.has_ecf=true
#cryptolib.tc.44.0.0.max_frame_length=1024
#cryptolib.tc.44.1.0.has_segmentation_header=false
#cryptolib.tc.44.1.0.has_pus_header=false
#cryptolib.tc.44.1.0.has_ecf=true
#cryptolib.tc.44.1.0.max_frame_length=1024
#cryptolib.tc.44.2.0.has_segmentation_header=false
#cryptolib.tc.44.2.0.has_pus_header=false
#cryptolib.tc.44.2.0.has_ecf=true
#cryptolib.tc.44.2.0.max_frame_length=1024
#cryptolib.tc.44.3.0.has_segmentation_header=false
#cryptolib.tc.44.3.0.has_pus_header=false
#cryptolib.tc.44.3.0.has_ecf=true
#cryptolib.tc.44.3.0.max_frame_length=1024

#Logging
logging.level.root=WARN
logging.level.gov.nasa.jpl.amos.asc.kmc=WARN

```

5.2.3.2.3 Configuring the SA Management GUI Service

To use the SA Management GUI, the `/opt/amos/kmc/services/sa-mgmt/etc/kmc-sa-mgmt.properties` configuration file must be edited to add the correct parameter values.

An example of the `kmc-sa-mgmt.properties` is shown below. In this example, mTLS is used to authentication to the SADB, which is the recommended configuration.

```
# KMC Security Association (SA) Management Properties File.
#
# See the KMC Product Guide for descriptions of each supported property.
#
# database user name
db.auth.user=admin
# database password
db.auth.pass=
# database connection string. if provided, the db.host, db.port, and db.schema keys will be
ignored
db.conn.string=none
# database host
db.host=sadb.example.com
# database port
db.port=3306
# database schema
db.schema=sadb
# database TLS keystore
db.keystore=/etc/pki/tls/private/amos-admin-keystore.p12
# database TLS keystore password
db.keystore.pass=changeit
# database TLS truststore
db.truststore=/etc/pki/tls/certs/amos-truststore.jks
# database TLS truststore password
db.truststore.pass=changeit
# use TLS for database connections
# the provided truststore must contain the certificate chain for connecting to the database.
db.tls=true
# use mutual TLS for database connections. when mTLS is true, the db.auth.pass key is ignored.
# the provided keystore must be trusted by the database server and database user.
db.mtls=true
```

Logging for the SA Management GUI Service is controlled by the `/opt/amos/kmc/services/sa-mgmt/etc/kmc-sa-mgmt-log4j2.xml` configuration file. An unmodified example of the file delivered with the DCS software is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
~ Copyright 2022, by the California Institute of Technology.
~ ALL RIGHTS RESERVED. United States Government Sponsorship acknowledged.
~ Any commercial use must be negotiated with the Office of Technology
~ Transfer at the California Institute of Technology.
~
~ This software may be subject to U.S. export control laws. By accepting
~ this software, the user agrees to comply with all applicable U.S.
~ export laws and regulations. User has the responsibility to obtain
~ export licenses, or other export authority as may be required before
~ exporting such information to foreign countries or providing access to
~ foreign persons.
-->
<Configuration status="ERROR">
  <Appenders>
    <RollingFile
      name="file"
      fileName="${sys:app.home}/logs/kmc-sa-mgmt.log"
      filePattern="${sys:app.home}/logs/kmc-sa-mgmt.%d{yyyy-MM-dd}.log.gz"
```

```

        ignoreExceptions="false">
        <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
        <Policies>
            <TimeBasedTriggeringPolicy interval="1"/>
        </Policies>
        <DefaultRolloverStrategy max="5"/>
    </RollingFile>
    <Console name="Console" target="SYSTEM_OUT">
        <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
</Appenders>
<Loggers>
    <Root level="ERROR">
        <!--          Uncomment to see log to console-->
        <!--          <AppenderRef ref="Console"/>-->
    </Root>
    <Logger name="gov" level="INFO" additivity="false">
        <AppenderRef ref="file"/>
    </Logger>
    <Logger name="org.hibernate" level="ERROR" additivity="false">
        <AppenderRef ref="file"/>
    </Logger>
</Loggers>

```

5.2.3.3 Starting and Stopping the DCS Services

After being properly configured, one or more of the DCS Services can be enabled and started as described below.

For each service to start automatically at boot up, use the corresponding command below:

```

$ sudo systemctl enable kmc-crypto-service
$ sudo systemctl enable kmc-sa-mgmt-service
$ sudo systemctl enable kmc-sdls-service

```

To prevent a service from automatically starting at boot up (i.e., when not in use, or as necessary when performing maintenance/upgrades) use the corresponding command below:

```

$ sudo systemctl disable kmc-crypto-service
$ sudo systemctl disable kmc-sa-mgmt-service
$ sudo systemctl disable kmc-sdls-service

```

The services can be started with the following commands:

```

$ sudo systemctl start kmc-crypto-service
$ sudo systemctl start kmc-sa-mgmt-service
$ sudo systemctl start kmc-sdls-service

```

The services can be stopped with the following commands:

```

$ sudo systemctl stop kmc-crypto-service
$ sudo systemctl stop kmc-sa-mgmt-service
$ sudo systemctl stop kmc-sdls-service

```

5.3 DCS SERVICE CONTAINER DEPLOYMENT

DCS provides the option to deploy the DCS services as containers. The containers are provided as OCI-compliant container images that are self-contained virtual environments. The DCS container images can be obtained from Artifactory at JPL, or from Multi-mission Configuration Management for non-JPL venues.

The deployment of containers into container engine environments varies widely from one environment to another. The instructions in this document below are for Podman because it is the container engine provided and supported by Red Hat.

The DCS Service Containers are designed to simplify as many of the configuration choices as possible, and provide self-contained DCS Services. In the interest of simplifying deployment of the DCS Service Containers, some configuration choices available to the full service have been preselected. First, the DCS Crypto Service Container is preconfigured to use a PKCS12 or BCFKS keystore for cryptographic materials, and not to use a KMS. The DCS Crypto Service Container is also preconfigured for mutual TLS (mTLS) authentication using x.509 keys and certificates, and does not support CAM for access control.

The DCS Service Containers can be deployed on a Linux system, or virtual system, with Docker or Podman. They can also be deployed in the Amazon Web Services (AWS) Elastic Cluster Service (ECS). Some environmental considerations need to be taken into account prior to deployment, however, depending on which deployment environment is to be used.

The first consideration is whether to also deploy a MariaDB container to host the Security Association Database (SADB), or to have the DCS Crypto Service connect to an existing database service for the SADB. It is **strongly** recommended that DCS Crypto Service Containers deployed in AWS ECS use an external SADB, either hosted on a Linux system, EC2 instance, or in the Amazon RDS service, due to the lack of the backing filestore for containers in ECS.

5.3.1 CONTAINER USER NAMESPACE MAPPING CONFIGURATION

Host-based containerization systems like Docker and Podman provide a User Namespace Mapping capability, which can be used to help isolate operations and file ownerships inside the container from those on the host system. Basically, this is accomplished with an offset to UIDs and GIDs, configured on the host system for each user that may be executing containers.

The namespace mapping configurations are in `/etc/subuid` and `/etc/subgid`, and both use the following format:

```
<user>:<offset>:<number_of_IDs>
```

The AMMOS standard configuration for Docker provides the following, for example:

```
dockremap:800000:65536
```

With this configuration, any UID inside a container using the docker remapping capability will be offset by 800000 – uid 0 (root) becomes 800000, uid 59111 (crypto) becomes 859111, etc.

The offsets for each user may be different in each deployment. Verify the correct UID/GID settings for configuration files and other artifacts that are needed by containerized applications at deployment time.

5.3.2 LOADING DCS SERVICE CONTAINERS

All of the container images that DCS has been tested with operate some version of an image repository – local storage for container images that can then be used to deploy virtualized application environments. Podman and Docker both have a subset of `image` commands to manage container images, while AWS ECS has a tight integration with the AWS Elastic Container Registry (ECR) service that performs the same function.

Each image repository has a capability that allows images to be imported from files, or uploaded from one repository to another. The image files generated in Section 3.6.3 can be transferred to the systems for deployment, and loaded to the local image repository with the following command:

```
$ podman image load -i <image_file>
```

For DCS 3.7.0, the images can all be loaded to the local repository with:

```
$ podman image load -i kmc-crypto-service-3.7.0.tar
$ podman image load -i kmc-sa-mgmt-service-3.7.0.tar
$ podman image load -i kmc-sdls-service-3.7.0.tar
```

Note: The commands noted above are included in the deployment instructions in the next subsection, and do not have to be repeated. Run each one once when deploying the containers, as described in the next subsection.

5.3.3 DEPLOYING THE DCS SERVICE CONTAINERS MANUALLY

The DCS service containers can be configured manually using podman (or docker) commands. The instructions in this subsection use podman because it is the default container service for RHEL9, which is the platform that the DCS software is tested on.

Compose tool (e.g., podman-compose or docker-compose) files for the DCS services are not included in the DCS 3.7.0 release. Providing such files is an objective for a future release.

The containers are configured with a set of secrets and environment variables, as described in the subsections below.

5.3.3.1 Enter Container Secrets

The DCS services use TLS to protect communications with the services. The commands shown below import the TLS keys, certificates, truststore, and CA bundle from files on the host. Before setting up the secrets in the container, deploy the TLS files as described in Section 4.1

The following commands are used to enter the TLS related secrets that are used by all 3 DCS service containers:

```
$ podman secret create tls_host_key /etc/pki/tls/private/ammos-server-key.pem
$ podman secret create tls_host_cert /etc/pki/tls/certs/ammos-server-cert.pem
$ podman secret create tls_ca_bundle /etc/pki/tls/certs/ammos-ca-bundle.crt
$ podman secret create tls_mtls_truststore /etc/pki/tls/private/ammos-mtls-truststore.jks
```

The container startup script will generate an `ammos-truststore.jks` file from the `ammos-ca-bundle.crt` file unless (optionally) an `ammos-truststore.jks` is passed into the container as follows:

```
$ podman secret create tls_truststore /etc/pki/tls/private/ammos-truststore.jks
```

The truststore password for ammos-truststore.jks will be set to `changeit` by default. To set a different truststore password, create a file (with proper permissions to prevent unauthorized access to it) that contains the truststore password (so that the password is not passed on the command line), and use it in the following command:

```
$ podman secret create tls_truststore_pass /path/to/truststorepwfile
```

The container startup script will automatically generate the host keystore file from the host key and cert files. There is no need to specifically set and use the `tls_host_keystore` secret, but it can be set (i.e., to override the automatic generation of the file) as follows:

```
$ podman secret create tls_host_keystore /etc/pki/tls/private/amos-server-keystore.bcfks
```

The example above shows a BCFKS keystore file (the recommended format), but a PKCS12 file can be used instead (unless the host will be operated in FIPS mode). Be sure that the keystore format matches the `KEYSTORE_FORMAT` setting.

The host keystore password default is 'changeit' and does not have to be provided unless a different password is desired. To set a different password, put it in a file and use it as follows:

```
$ podman secret create tls_host_keystore_pass /path/to/hostkeystorepass
```

The DCS Crypto Service container uses a Crypto Keystore to store cryptographic keys used for encryption, decryption, and integrity checking. Prior to setting the Crypto Keystore secrets, create the Crypto Keystore and an associated keystore password file, as described in Section 4.1.3. A BCFKS keystore type is recommended.

The Crypto Keystore and keystore password secrets are set by the following commands:

```
$ podman secret create crypto_keystore /path/to/crypto_keystore.bcfks
$ podman secret create crypto_keystore_pass /path/to/keystorepass
```

The SDLS Service and the SA Management GUI Service use the SADB. The recommendation and default configuration is to use mTLS authentication to the SADB, but it can be set up for password authentication. If the SADB is set up to use password authentication, place the SADB user password into a file, set the secret as follows, and pass it to the `podman container create` command.

```
$ podman secret create db_user_pass /path/to/dbuserpass
```

5.3.3.2 Starting the Crypto Service Container

The Crypto Service container uses some of the secrets covered in Section 5.3.3.1, as shown in the example `podman container create` command shown further down.

The following environment variables may be passed to the container (using the `-e` option in the `podman` commands) when it is created. Both have a default and are optional. Determine the correct values for the variables before starting the container.

- `JAVA_MIN_HEAP`: Minimum memory heap for Tomcat JVM. Optional. Default is 1g.
 - For example: `JAVA_MIN_HEAP=2g`
- `JAVA_MAX_HEAP`: Maximum memory heap for Tomcat JVM. Increase the value as needed to increase performance under heavy load. Optional. Default is 2g.
 - For example: `JAVA_MAX_HEAP=4g`

The commands below start up the Crypto Service Container by performing the following actions:

- Load image from tarball
- Create the kmc-crypto-service container
- Create SystemD unit file for service
- Add firewall rule for inbound traffic for the kmc-crypto-service
- Enable/start the service & container

```
$ podman image load -i kmc/services/sa-mgmt-3.7.0.tar.gz
$ podman container create -p 8443:8443 --name kmc/services/sa-mgmt --secret
tls_host_key --secret tls_host_cert --secret tls_ca_bundle --secret crypto_keystore --
secret crypto_keystore_pass --secret tls_mtls_truststore kmc/services/sa-mgmt:3.7.0
$ podman generate systemd -n -f --container-prefix='pd' kmc/services/sa-mgmt
$ mv pd-kmc/services/sa-mgmt.service /etc/systemd/system/
$ chmod 644 /etc/systemd/system/pd-kmc/services/sa-mgmt.service
$ firewall-cmd --add-service=kmc/services/sa-mgmt
$ firewall-cmd --add-service=kmc/services/sa-mgmt --permanent
$ systemctl daemon-reload
$ systemctl enable --now pd-kmc/services/sa-mgmt
```

Note that some of the secrets covered in Section 5.3.3.1 are not used in the example above. To include the optional secrets, add them to the `podman container create` command by using the `--secret` option (e.g., `--secret tls_truststore_pass`).

The Crypto Service Container may take up to 90 seconds to initialize fully and come online. After that time, the Crypto Service should be available at `https://<CRYPTO_FQDN>:8443/` (e.g. `https://kmc.example.com:8443/crypto-service`).

5.3.3.3 Starting the SDLS Service Container

The SDLS Service relies on the MariaDB SADB. Set up the SADB, as described in Section 5.4, before starting the SDLS Service Container.

The SDLS Service also relies on TLS artifacts, which are passed into the container as secrets. See Section 5.3.3.1 for instructions about setting the secrets

The following environment variables are passed to the container when it is created. Some of them are optional, as noted in those cases. Determine the correct values before starting the container.

- **SADB_FQDN**: The fully-qualified domain name for the system hosting the SADB. This *can* be the same host that runs the SDLS Service Container, if MariaDB is configured and running on the host, or in another container.
 - For example: `SADB_FQDN=sadb.example.com`
- **CRYPTO_FQDN**: The fully-qualified domain name for the system hosting the DCS Crypto Service container.
 - For example: `CRYPTO_FQDN=crypto.example.com`
- **DB_USER**: The username that the SDLS Service will use to connect to the SADB. Optional. Default is `crypto`.
 - For example: `DB_USER=sadb-user`
 - Also, see Section 5.3.3.1 for instructions about the `db_user_pass` secret for password authentication to the SADB (which is a supported option, but not recommended).

- **CREATE_ECF:** Set to 'true' if DCS should create the ECF or 'false' if DCS should not create the ECF. Optional. Default is 'true'.
 - For example: `CREATE_ECF='true'`
- **VCID_BITMASK:** The bitmask to apply to the VCID. Optional. Default is '0x3f'.
 - For example: `VCID_BITMASK='0x3f'`
- **IGNORE_ANTIPLAY:** Set to 'true' if DCS should ignore anti-replay counters (e.g., in a test environment), or 'false' if DCS should not ignore anti-replay. Optional. Default is 'true'.
 - For example: `IGNORE_ANTIPLAY='true'`
- **IGNORE_SA_STATE:** Set to 'true' if DCS should ignore the state SAs (e.g., in a test environment), or 'false' if DCS should not ignore the state SAs. Optional. Default is 'true'.
 - For example: `IGNORE_SA_STATE='true'`
- **CHECK_FECF:** Set to 'true' if DCS should check the ECF (and return an error if the ECF is incorrect) or 'false' if DCS should not check the ECF. Optional. Default is 'false'.
 - For example: `CHECK_FECF='false'`
- **MANAGED_PARAMS:** The managed parameters described in Section 4.2.3.1. Place the managed parameters in a file and import the parameters as shown in the example below.
 - For example: `MANAGED_PARAMS="cat /path/to/sdls_managed_params"`
- **JAVA_MIN_HEAP:** minimum memory heap for Tomcat JVM. Optional. Default is 1g.
 - For example: `JAVA_MIN_HEAP=2g`
- **JAVA_MAX_HEAP:** maximum memory heap for Tomcat JVM. Optional. Default is 2g.
 - For example: `JAVA_MAX_HEAP=4g`

The commands below start up the SDLS Service Container by performing the following actions:

- Loads image from tarball
- Creates the kmc-sdls-service container
- Creates SystemD unit file for service
- Add firewall rule for inbound traffic for the kmc-sdls-service
- Enable/start the service & container

```
$ podman image load -i kmc-sdls-service-3.7.0.tar.gz
$ podman container create -p 8445:8445 --name kmc-sdls-service --secret tls_host_key --secret tls_host_cert --secret tls_ca_bundle --secret tls_mtls_truststore -e CRYPTO_FQDN=crypto.example.com -e SADB_FQDN=sadb.example.com -e MANAGED_PARAMS="cat /path/to/sdls_managed_params" kmc-sdls-service:3.7.0
$ podman generate systemd -n -f --container-prefix='pd' kmc-sdls-service
$ mv pd-kmc-sdls-service.service /etc/systemd/system/
$ chmod 644 /etc/systemd/system/pd-kmc-sdls-service.service
$ firewall-cmd --add-service=kmc-sdls-service
$ firewall-cmd --add-service=kmc-sdls-service --permanent
$ systemctl daemon-reload
$ systemctl enable --now pd-kmc-sdls-service
```

The SDLS Service Container may take up to 90 seconds to initialize fully and come online. After that time, the SDLS Service should be available at https://<CRYPTO_FQDN>:8445/ (e.g., <https://kmc.example.com:8445/sdls-service>).

5.3.3.4 Starting the SA Management GUI Service Container

The SA Management GUI relies on a MariaDB SADB. Set up the SADB, as described in Section 5.4 before starting the SA Management GUI Service Container.

The SDLS Service also relies on TLS artifacts, which are passed into the container as secrets. See Section 5.3.3.1 for instructions about setting the secrets

The following environment variables are passed to the container when it is created. Some of them are optional, as noted in those cases. Determine the correct values for the variables before starting the container.

- **SADB_FQDN:** The fully-qualified domain name for the system hosting the SADB. This *can* be the same host that runs the SA Management Service Container, if MariaDB is configured and running on the host, or in another container.
 - For example: SADB_FQDN=sadb.example.com
- **DB_USER:** The username that the SA Management GUI service will use to connect to the SADB. Optional. Default is crypto.
 - For example: DB_USER=sadb-user
 - Also, see Section 5.3.3.1 for instructions about the db_user_pass secret for password authentication to the SADB (which is a supported option, but not recommended).
- **DB_MTLS:** Set to 'true' for the SA Management GUI Service to use mTLS to authenticate to the SADB, or set to 'false' to not use mTLS to the SADB. Optional. Default is 'true'.
 - For example: DB_MTLS='true'
- **KEYSTORE_FORMAT:** The format of the host (i.e., server) keystore, which is used to authenticate to the SADB when mTLS is used. Can be BCFKS (recommended) or PKCS12. Optional. Default is BCFKS.
 - For example: KEYSTORE_FORMAT=BCFKS
 - Also, see Section 5.3.3.1 for instructions about the tls_host_keystore and tls_host_keystore_pass secrets that are related to this keystore type variable.
- **JAVA_MIN_HEAP:** Minimum memory heap for Tomcat JVM. Optional. Default is 1g.
 - For example: JAVA_MIN_HEAP=2g
- **JAVA_MAX_HEAP:** Maximum memory heap for Tomcat JVM. Optional. Default is 2g.
 - For example: JAVA_MAX_HEAP=4g

The commands below start up the SA Management Service Container by performing the following actions:

- Loads image from tarball
- Creates the kmc-sdls-service container (uses the same secrets as the Crypto Service Container, but not all of them)
- Creates SystemD unit file for service
- Add firewall rule for inbound traffic for the kmc-sdls-service
- Enable/start the service & container

```
$ podman image load -i kmc-sa-mgmt-service-3.7.0.tar.gz
$ podman container create -p 8447:8447 --name kmc-sa-mgmt-service --secret
tls_host_key --secret tls_host_cert --secret tls_ca_bundle --secret
tls_mtls_truststore -e SADB_FQDN=sadb.example.com kmc-sa-mgmt-service:3.7.0
$ podman generate systemd -n -f --container-prefix='pd' kmc-sa-mgmt-service
$ mv pd-kmc-sa-mgmt-service.service /etc/systemd/system/
$ chmod 644 /etc/systemd/system/pd-kmc-sa-mgmt-service.service
$ firewall-cmd --add-service=kmc-sa-mgmt-service
$ firewall-cmd --add-service=kmc-sa-mgmt-service --permanent
$ systemctl daemon-reload
$ systemctl enable --now pd-kmc-sa-mgmt-service
```

The SA Management Service Container may take up to 90 seconds to initialize fully and come online. After that time, the SA Management Service should be available at https://<CRYPTO_FQDN>:8447/ (e.g. <https://kmc.example.com:8447/sa-mgmt>).

5.4 DEPLOYING THE SECURITY ASSOCIATION DATABASE (SADB)

AMMOS Command Encryption capabilities in DCS are developed and testing using the MariaDB database (as the SADB) that comes with the RHEL 9 operating system. DCS deployments that will use the Command Encryption capabilities require that an instance of the SADB is installed – that instance may be on the same host as any of the DCS software components, or on a separate, network-accessible system.

5.4.1 DEPLOYING MARIADB ON THE DATABASE HOST

Verify that the MariaDB software is installed on the system intended to host the SADB instance. On RHEL 9 systems, the following packages can be installed to provide all of the necessary software:

- mariadb-connector-c-config
- mariadb-common
- mariadb-errmsg
- mariadb-connector-c
- mariadb
- mariadb-backup
- mariadb-gssapi-server
- mariadb-server-utils
- mariadb-server

DCS 3.7 has been tested with both MariaDB 10.5 (10.5.22) and MariaDB 10.11 (10.11.6), but should be broadly compatible with any MariaDB version 10.4 and greater. Note: the configuration steps described below are targeted to RHEL 9 installations of MariaDB – configuration files and options might vary on other platforms.

The steps in this subsection can be followed to set up the database. Once configured, the kmc-sa-mgmt CLI described at the end of this subsection can be used to list, create, update, delete, start, stop, reset, and rekey SAs.

MariaDB must be configured to support TLS network connections from the other DCS software components – which requires at least a TLS Server key and certificate. See Section 4.1.1.1 for details on how to generate these artifacts, and Section 5.2.2.1 for installation instructions. The steps for configuring TLS support and running the initial security script are only necessary for *new* MariaDB installations – if adding the SADB component to an existing MariaDB server instance, skip ahead to Section 5.4.2.

To enable TLS network connectivity in MariaDB, edit `/etc/my.cnf`, and add the following lines at the end of the `[mysqld]` section. This step requires elevated privileges (root, sudo privileges, etc.).

```
ssl_cert=/etc/pki/tls/certs/amos-server-cert.pem
ssl_key=/etc/pki/tls/private/amos-server-key.pem
ssl_ca=/etc/pki/tls/certs/amos-ca-bundle.crt
bind-address=0.0.0.0
ssl=true
```

The `bind-address` parameter above will allow for incoming connections on ALL IPv4 network interfaces available on the host system. To constrain incoming connections to a specific interface, replace `0.0.0.0` with the specific IP address to allow connections on.

See Section 4.1.1 for information about, and examples of, how to generate TLS Server key & certificate pairs. The settings above use the AMMOS default settings for the files. Once the configuration file has been updated, restart the MariaDB service with elevated privileges:

```
$ sudo systemctl restart mariadb
```

MariaDB is not configured securely in the default state. Make sure to run the `mysql_secure_installation` script as root to set the database root user password, eliminate test databases that are often misused, and limit administrative access to the database:

```
$ /usr/bin/mysql_secure_installation
```

The `mysql_secure_installation` script will prompt you to perform the steps above, and other than the password for the database root user, the defaults provided are reasonable and safe.

TLS capabilities can be confirmed after the database restart with the following command:

```
$ echo "show variables like '%ssl%';" | mysql -u root -p
```

After prompting you for the MariaDB root password, the command should output the state of SSL variables. The values will show that the key and certificate settings added to the `mariadb-server.cnf` config file above match intended settings, and TLS connectivity can be verified with the “HAVE_SSL” variable:

```
Enter password:
Variable_name Value
have_openssl YES
have_ssl YES
ssl_ca /etc/pki/tls/certs/amos-ca-bundle.crt
ssl_capath
ssl_cert /etc/pki/tls/certs/amos-server-cert.pem
ssl_cipher
kEECDH:+kEECDH+SHA:kEDH:+kEDH+SHA:+kEDH+CAMELLIA:kECDH:+kECDH+SHA:kRSA:+kRSA+SHA:+
kRSA+CAMELLIA:!aNULL:!eNULL:!SSLv2:!RC4:!DES:!EXP:!SEED:!IDEA:!3DES
ssl_crl
ssl_crlpath
ssl_key /etc/pki/tls/private/amos-server-key.pem
version_ssl_library OpenSSL 3.2.2 4 Jun 2024
```

5.4.2 CREATING THE SADB SCHEMA

The DCS Crypto Client contains the following SQL script, which creates the schema in the SADB.

- /opt/ammos/kmc/etc/sa_mariadb_sql/create_sadb.sql

Execute the create_sadb.sql script to create the SADB SQL tables as follows:

```
$ mysql -u root -p < /opt/ammos/kmc/etc/sa_mariadb_sql/create_sadb.sql
```

The create_sadb.sql script is composed of the following SQL:

```
CREATE DATABASE IF NOT EXISTS sadb;

USE sadb;

-- IV_LEN should probably not have that default -- to be reviewed.

CREATE TABLE IF NOT EXISTS security_associations
(
    spi INT NOT NULL
    , ekid VARCHAR(100) CHARACTER SET utf8 DEFAULT NULL -- 'EG, for KMC Crypto KeyRef,
'kmc/test/KEY130', for libgcrypt '130'
    , akid VARCHAR(100) CHARACTER SET utf8 DEFAULT NULL -- Same as ekid
    , sa_state SMALLINT NOT NULL DEFAULT 0
    , tfvn TINYINT NOT NULL
    , scid SMALLINT NOT NULL
    , vcid TINYINT NOT NULL
    , mapid TINYINT NOT NULL DEFAULT 0
    , lpid SMALLINT
    , est SMALLINT NOT NULL DEFAULT 0
    , ast SMALLINT NOT NULL DEFAULT 0
    , shivf_len SMALLINT NOT NULL DEFAULT 0
    , shsnf_len SMALLINT NOT NULL DEFAULT 0
    , shplf_len SMALLINT NOT NULL DEFAULT 0
    , stmacf_len SMALLINT NOT NULL DEFAULT 0
    , ecs_len SMALLINT NOT NULL DEFAULT 1
    , ecs VARBINARY(4) NOT NULL DEFAULT X'01' -- ECS_SIZE=4
    , iv_len SMALLINT NOT NULL DEFAULT 0
    , iv VARBINARY(20) DEFAULT NULL -- IV_SIZE=12
    , acs_len SMALLINT NOT NULL DEFAULT 0
    , acs VARBINARY(4) NOT NULL DEFAULT X'00'
    , abm_len MEDIUMINT
    , abm VARBINARY(1024) NOT NULL DEFAULT X'0000FC0000FFFF000000000000000000000000' --
ABM_SIZE=1024
    , arsn_len SMALLINT NOT NULL DEFAULT 0
    , arsn VARBINARY(20) NOT NULL DEFAULT X'000000000000000000000000000000000000' --
ARSN_SIZE=20 , TBD why so large...
    , arsnw SMALLINT NOT NULL DEFAULT 0 -- ARSNW_SIZE=1
);

create unique index if not exists main_spi on security_associations
(spi,scid,vcid,tfvn,mapid);
```

After the script has been processed, MariaDB should now have an sadb database, containing an empty security_associations table. Verify that this step has been completed with the following MariaDB commands by connecting to the MariaDB admin interface with the following command and supplying the MariaDB root password:

```
$ mysql -u root -p
```

In the example below, the commands to be entered start after the greater-than (>) symbol – everything prior to and including that symbol is the prompt from the mysql client.

```
MariaDB [(none)]> show databases like '%sadb%';
+-----+
| Database (%sadb%) |
+-----+
| sadb               |
+-----+
1 row in set (0.001 sec)
```

MariaDB [(none)]> use sadb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

```
MariaDB [sadb]> show tables;
+-----+
| Tables_in_sadb      |
+-----+
| security_associations |
+-----+
1 row in set (0.000 sec)
```

```
MariaDB [sadb]> select count(spi) from security_associations;
+-----+
| count(spi) |
+-----+
|          0 |
+-----+
1 row in set (0.025 sec)
```

5.4.3 CREATING MARIADB USER ACCOUNTS

MariaDB instances manage user accounts for database access separately from the operating system user accounts. For DCS deployments, a minimum of three MariaDB user accounts are required:

- A non-privileged user account, named `client` in these examples, for using command encryption activities but not managing SAs. This user is constrained by database permissions, and can only change the `iv` and `arsn` fields of the `security_associations` table.
- A privileged admin user account, named `admin` in these examples, primarily used by the `kmc-sa-mgmt` tool for managing security associations. This user is allowed to modify all elements of the `security_associations` table.
- An account for DCS Service access, named `cryptosvc` in these examples, which will be used by the DCS SDLs Service and DCS SA Management GUI to access the SADB. This user is allowed to modify all elements of the `security_associations` table.

These account names are largely arbitrary, and can be changed as desired in customized deployments. Make sure to account for database user name changes in database and DCS software configuration steps. Additional accounts, privileged and non-privileged, can be created as needed for DCS deployments, but the three accounts listed above are required for the DCS software components to interoperate correctly.

Network access to MariaDB adds another variable to the database user configuration steps. In MariaDB, every combination of *username@fqdn* is considered a unique user (unless using wildcards to configure user accounts, which is not recommended). When planning MariaDB database user accounts, plan for the following database access needs:

- For each system running the DCS Crypto Client, a MariaDB client user must be created with the FQDN for the systems running the Crypto Client: `client@kmc-client1.example.com`, `client@kmc-client2.example.com`, etc. Use the client certificates created in Section 4.1.1.2 and installed in Section 5.2.2.1 for mTLS authentication of Crypto Client users. Note: it *is not* required for the client certificate subjects to match each of the hostnames that admin users will be connecting from – rather, it's important that the database user and the certificate that will be presented by the client agree on the contents of the certificate subject.
- For each system allowed to manage the SADB, which can be accomplished via the SA Management GUI *service* and the `kmc-sa-mgmt` command line interface included in the DCS Crypto Client, a MariaDB admin user must be created with the FQDN for that system: `admin@kmc-client1.example.com`, `admin@kmc-admin1.example.com`, etc. It is *not required* to allow administrative access from every system running the DCS Crypto Client – just from those that are intended for administrative use. Admin users should use the admin certificates created in Section 4.1.1.2 and installed in 5.2.2.1 for mTLS authentication. Note: it *is not* required for the admin certificate subjects to match each of the hostnames that admin users will be connecting from – rather, it's important that the database user and the certificate that will be presented by the client agree on the contents of the certificate subject.
- For each system that will be running the DCS SDLS Service *or* the DCS SA Management GUI, a MariaDB admin user must be created with the FQDN for that system: `cryptosvc@kmc.example.com` (system running all of the DCS components), `cryptosvc@sdls.example.com` (system running just the SDLS service), `cryptosvc@samgmt.example.com` (system running just the SA Management GUI), etc. Use the TLS System certificate for mTLS authentication of DCS Service users.

Once the list of MariaDB user accounts to be created has been compiled, additional information must be gathered prior to executing the database commands. Each of the user accounts must be secured by either a password, which is stored in the database as a specially hashed string, or through the use of mutual TLS authentication via a TLS certificate. User accounts *can* be configured with both a password and mutual TLS authentication, should that be desired, but such a configuration does not add much to the security posture of an individual database user account. It is ***strongly*** recommended that the MariaDB database accounts for SADB access utilize mutual TLS authentication, with separate TLS key and certificate pairs for the users.

For each SADB user utilizing mTLS authentication, the subject string needs to be extracted from the mTLS client certificate prior to database setup. This string can be extracted from the certificate file with the following command, replacing `<mtls_certificate_file>` with the correct filename for the mTLS client certificate:

```
$ /bin/openssl x509 -in <mtls_certificate_file> -noout -subject | cut -c9- | \
  sed -e 's/ = /=/g' -e 's/, /\//g' -e 's/^/\\/'
```

That command will return formatted Subject string, required to configure mTLS authentication in the database, similar to the example below:

```
/C=US/ST=California/L=Pasadena/O=Example
Org/OU=Example/OU/CN=SADB_testuser/emailAddress=email@example.com
```

Connect to the SADB administrative interface using the following command, supplying the MariaDB root password:

```
$ mysql -u root -p
```

Perform the following steps to create each desired SADB account and grant to it permissions to the SA tables. Important note: the hostname provided in the command *must* be the fully qualified domain name (FQDN) of the system that the user will be connecting *from*. In environments where DCS software components are distributed across multiple systems, make sure that the correct username and hostname combinations are granted access. The username, fully qualified host name, and TLS subject string should be enclosed in single quotes.

To create a database user with mTLS authentication, adjust the command below as needed:

```
create user '<username>'@'<fqdn>' require subject '<TLS Subject>';
```

For example (all on one line):

```
create user 'user'@'cmdenc-test-client.example.com' require subject
'/C=US/ST=California/L=Pasadena/O=Example Org/OU=Example
OU/CN=SADB_testuser/emailAddress=email@example.com';
```

To create a database user with password authentication, adjust the command below as needed:

```
create user '<username>'@'<fqdn>' identified by '<password>';
```

For example:

```
create user 'user2'@'cmdenc-test-client.example.com' identified by 'l0ngp@ssphr4se';
```

Once user accounts have been created, the correct permissions need to be granted. For administrative and service users, issue the following permission grant:

```
grant all privileges on sadb.* to '<user>'@'<fqdn>';
```

For example:

```
grant all privileges on sadb.* to 'adminuser'@'cmdenc-test-client.example.com';
grant all privileges on sadb.* to 'cryptosvc'@'cmdenc-test-client.example.com';
```

For nonprivileged users, issue the following permission grants:

```
GRANT UPDATE (arsn) on sadb.security_associations to '<user>'@'<fqdn>';
GRANT UPDATE (iv) on sadb.security_associations to '<user>'@'<fqdn>';
GRANT SELECT on sadb.security_associations to '<user>'@'<fqdn>';
```

For example:

```
GRANT UPDATE (arsn) ON sadb.security_associations TO 'user'@'cmdenc-test-
client.example.com';
GRANT UPDATE (iv) ON sadb.security_associations TO 'user'@'cmdenc-test-
client.example.com';
GRANT SELECT ON sadb.security_associations TO 'user'@'cmdenc-test-client.example.com';
```

Once all the desired SADB users have been added, run the following command to update the database privilege settings:

```
flush privileges;
```

A mysql command can be run to verify that mTLS authentication for each user is configured correctly. The mysql command does not involve cryptolib or kmc-sa-mgmt, and so it provides a good test of the keys/certs. Adjust the command below as needed to verify database connectivity for each of the SADB users:

```
$ echo 'show tables' | mysql -u <user> -h <sadb_fqdn> \
--ssl-ca=/etc/pki/tls/certs/amos-ca-bundle.crt --ssl-verify-server-cert \
--ssl-cert=<path to certificate file> \
--ssl-key=<path to key file> sadb
```

For example, if the database is running on cmdenc-test-client.example.com:

```
$ echo 'show tables' | mysql -u user -h cmdenc-test-client.example.com \
--ssl-ca=/etc/pki/tls/certs/amos-ca-bundle.crt --ssl-verify-server-cert \
--ssl-cert=/etc/pki/tls/certs/amos-client-cert.pem \
--ssl-key=/etc/pki/tls/private/amos-client-key.pem sadb
```

The command will return the output of the 'show tables' command, which should look like the result below:

```
Tables_in_sadb
security_associations
```

Repeat the verification command for each database user created, from the configured host.

5.4.4 LOADING SECURITY ASSOCIATION DATA

DCS provides a command line interface (CLI) tool for loading SAs from a project-specific file (as described in Section 4.3.1). Prior to running the commands below, make sure to configure the kmc-sa-mgmt utility as described in Section 5.2.3.1.

After configuration, use the kmc-sa-mgmt utility along with the project-specific SA file (e.g., project_SAs.csv) as follows to create SAs in the database. Make sure to adjust the paths to the kmc-sa-mgmt tool for customized deployments and to the CSV file containing security association data created in Section 4.3.1.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt create --file=project_SAs.csv
```

The command above will add new SAs to the SADB, and skip any existing SAs that are listed in the CSV file. To update existing SAs, execute the following command:

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt update --file=project_SAs.csv
```

The same CSV file can be used to create and to update SAs (but not delete SAs). Any SAs not already in the database will be added (when the "create" subcommand is used), and any pre-existing SAs will be updated (when the "update" subcommand is used).

The create and update subcommands for kmc-sa-mgmt do not check for conflicts, such as two or more active SAs for the same GVCID. When creating SAs, the tool will make sure that there are not duplicate SPI numbers for each GVCID, and some other basic checking of the input. To avoid SA conflicts, make sure that the CSV file does not specify more than one active SA for each GVCID, and use the start/stop/expire subcommands to change SA state (rather than using "update" to change SA state).

The /opt/amos/kmc/bin/kmc-sa-mgmt command line interface (CLI) used in the command above provides abilities for managing SAs in the SADB. See Section 6.1 for instructions for using the CLI to manage SAs.

5.5 VALIDATING DCS DEPLOYMENTS

Once the deployment of the various DCS components is complete, there are several steps that can be used to validate the configuration and that all the services are interacting as expected.

5.5.1 CHECKING DCS SERVICE STATUS ENDPOINTS

Each of the DCS services (crypto-service, sdls-service, and sa-mgmt) have status endpoints that can be accessed with a browser or a command-line client like curl to verify that the service basics are up and running. For systems configured with mutual TLS authentication, one of the acceptable pairs of TLS key and cert must be configured in the browser or CLI client prior to connecting.

To check the DCS Crypto Service, use a browser or CLI client to visit the following endpoint, after adjusting the hostname and port as needed:

```
https://kmc.example.com:8443/crypto-service/status
```

To check the DCS SDLS Service, use a browser or CLI client to visit the following endpoint, after adjusting the hostname and port as needed:

```
https://kmc.example.com:8445/sdls-service/status
```

To check the DCS SA Management Service, use a browser or CLI client to visit the following endpoint, after adjusting the hostname and port as needed:

```
https://kmc.example.com:8447/sa-mgmt/api/status
```

5.5.2 USING THE DCS SDLS TESTING TOOL

The DCS Crypto Client includes a simple test tool that can be used to validate end-to-end SDLS encryption and decryption. It uses the DCS SDLS Python API (which goes through the DCS SDLS C API to CryptoLib) to encrypt and then decrypt a test frame.

The tool is installed in `/opt/ammos/kmc/test/bin/kmc_sdls_test_app.py`, and is configured in `/opt/ammos/kmc/test/etc/kmc_sdls_test_app.properties`.

To configure the test tool, edit the `/opt/ammos/kmc/test/etc/kmc_sdls_test_app.properties` file. An example of the properties file is shown below.

```
#Sample DCS SDLS Properties File
#See the DCS SIS for descriptions of each supported property.

cryptolib.sadb.type=mariadb
#cryptolib.sadb.mariadb.fqdn=cryptol23.example.com
cryptolib.sadb.mariadb.fqdn=sa-host.example.com
cryptolib.sadb.mariadb.username=test-user
#Password not necessary when using mTLS
cryptolib.sadb.mariadb.password=
cryptolib.sadb.mariadb.tls.cacert=/etc/pki/tls/certs/ammos-ca-bundle.crt
cryptolib.sadb.mariadb.tls.verifyserver=true
cryptolib.sadb.mariadb.mtls.clientcert=/etc/pki/tls/certs/test-user-cert.pem
cryptolib.sadb.mariadb.mtls.clientkey=/etc/pki/tls/private/test-user-key.pem
cryptolib.sadb.mariadb.require_secure_transport=true

cryptolib.crypto.type=kmccryptoservice
cryptolib.crypto.kmccryptoservice.protocol=https
#cryptolib.crypto.kmccryptoservice.fqdn=cryptol23.example.com
cryptolib.crypto.kmccryptoservice.fqdn=crypto.example.com
```

```

cryptolib.crypto.kmccryptoservice.app=crypto-service
cryptolib.crypto.kmccryptoservice.mtls.clientcert=/etc/pki/tls/certs/test-user-cert.pem
cryptolib.crypto.kmccryptoservice.mtls.clientcertformat=PEM
cryptolib.crypto.kmccryptoservice.mtls.clientkey=/etc/pki/tls/private/test-user-key.pem
cryptolib.crypto.kmccryptoservice.cacert=/etc/pki/tls/certs/amos-ca-bundle.crt
cryptolib.crypto.kmccryptoservice.verifyserver=true

#CryptoLib Global Behavioral Properties
cryptolib.apply_tc.create_ecf=true
cryptolib.tc.vcid_bitmask=0x3f
cryptolib.process_tc.ignore_antireplay=true
cryptolib.process_tc.ignore_sa_state=true
cryptolib.process_tc.process_pdus=false
cryptolib.process_tc.check_fecf=false

#Managed Parameters
#cryptolib.tc.<scid>.<vcid>.<tfvn>.<property>
cryptolib.tc.44.0.0.has_segmentation_header=false
cryptolib.tc.44.0.0.has_pus_header=false
cryptolib.tc.44.0.0.has_ecf=true
cryptolib.tc.44.0.0.max_frame_length=1024
cryptolib.tc.44.1.0.has_segmentation_header=false
cryptolib.tc.44.1.0.has_pus_header=false
cryptolib.tc.44.1.0.has_ecf=true
cryptolib.tc.44.1.0.max_frame_length=1024
cryptolib.tc.44.2.0.has_segmentation_header=false
cryptolib.tc.44.2.0.has_pus_header=false
cryptolib.tc.44.2.0.has_ecf=true
cryptolib.tc.44.2.0.max_frame_length=1024
cryptolib.tc.44.3.0.has_segmentation_header=false
cryptolib.tc.44.3.0.has_pus_header=false
cryptolib.tc.44.3.0.has_ecf=true
cryptolib.tc.44.3.0.max_frame_length=1024

```

The test script requires that the SADB be available and populated with the Security Association(s) needed for the testing to be performed, and that the cryptographic keys referenced in the SA(s) are available in a Crypto Keystore or KMS that is accessible by the DCS Crypto Service.

The `/opt/amos/kmc/test/etc/kmc_sdls_test_app.properties` file must contain correct configuration settings for the SADB and DCS Crypto Service, the global behavioral properties, and managed parameter lines for the SCID(s)/VCID(s) to be used in testing.

The test tool also requires that the PYTHONPATH environment variable be set correctly to include all of the necessary Python libraries. At a minimum, the PYTHONPATH should include the following directory:

```
$ export PYTHONPATH=/opt/amos/kmc/lib/python3.9/site-packages
```

The test tool can use a default (built-in) TC frame for SCID 44, VCID 1, TFVN 0. The test tool accepts `-s` and `-V` arguments that can be used to alter the SCID and/or VCID in the default frame. Alternatively, the test tool accepts a `-f` argument to allow a custom frame to be used.

The test tool accepts a `-p` argument that can be used to specify a different properties file than the default path to the `/opt/amos/kmc/test/etc/kmc_sdls_test_app.properties` file.

The usage for the test tool is as follows:

```
$ ./kmc_sdls_test_app.py -h
```

```
usage: kmc_sdls_test_app.py [-h] [-f FRAME] [-p PROPERTIES] [-P] [-A]
                             [-s SCID] [-V VCID]
```

Simple KMC SDLS Python Test Application that will Apply and Process Security on a given frame

optional arguments:

```
-h, --help                show this help message and exit
-f FRAME, --frame FRAME   Hex frame string representation of telecommand
                           transfer-frame to apply & process SDLS layering on.
                           Defaults to: '202c0408000001bd37'
-p PROPERTIES, --properties PROPERTIES
                           The properties file that contains all the
                           configuration needed by this application (supported
                           properties defined in KMC SIS)
-P, --processOnly          Flag to only process security on the frame (default is
                           to apply & process)
-A, --applyOnly            Flag to only apply security on the frame (default is
                           to apply & process)
-s SCID, --scid SCID      Override the default frame SC ID field
-V VCID, --vcid VCID      Override the default frame VC ID field
```

Execution of the test script shows the test frame constructed for the test, the resulting frame after SDLS is applied, and the security header field values after the encrypted test frame has been SDLS Processed (decrypted).

```
$ ./kmc_sdls_test_app.py
Using telecommand transfer frame:
202c0408000001bd37
```

[illegible]

```
SDLS TC Process Security Result:
SPI: 2
IV: 00000000000000000000000000000009
PDU: 0001
MAC: 7475f40b902cdd53b3886fbbd73ed70b
FECE: 0x0
```

5.6 TROUBLESHOOTING DEPLOYMENT ISSUES

There are several steps that can be used to help diagnose problems with DCS deployments. The most common set of deployment issues are related to the TLS artifacts – keys, certificates, CA bundles, keystores and truststores, and their various formats. When encountering issues with TLS artifacts, it is common to see error messages like “Could not open key” or “Hostname/SNI did not match certificate,” and many others. Verify that all of the keys and certificates have the correct hostnames, the Not Before and Not After fields are currently valid, and that the trust anchors for the certificates are listed in the CA certificate bundles and the truststores in use.

PEM-formatted certificates can be queried for information using openssl:

```
$ openssl x509 -in /etc/pki/tls/certs/ammos-server-cert.pem -noout -text | head -11
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            53:00:01:04:fb:de:ec:0e:b1:af:7e:d5:07:00:00:00:01:04:fb
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: DC = com, DC = example, CN = Example.com Issuing Cert
        Validity
            Not Before: Feb  8 05:26:18 2024 GMT
            Not After : Feb  7 05:26:18 2026 GMT
        Subject: C = US, ST = California, L = Pasadena, O = Example.com, OU = ITDept,
            CN = kmc.example.com, emailAddress = kmc_list@example.com
```

The relevant certificate information is all in the output from openssl. Check that the CN in the Subject matches the DNS hostname for the system the cert is being used on. Check the timestamps in the Not Before and Not After fields. Make sure that the trust bundles (ammos-ca-bundle.crt and ammos-truststore.jks) contain the *Issuer's Certificate, and any CA certificates needed to validate the Issuer's Certificate*. For certificates used for mutual TLS, like the ammos-admin-cert.pem and ammos-client-cert.pem, the key thing to check is that the Subject being checked matches on both ends – the subject in the certificate file and the subject in the SADB user configuration, for example.

Openssl can also connect to a running service to retrieve the certificate that the service offers:

```
$ openssl s_client -connect kmc.example.com:8443 -showcerts
CONNECTED(00000003)
depth=2 CN = JPLICA Root
verify return:1
depth=1 DC = com, DC = example, CN = Example.com Issuing Cert
verify return:1
depth=0 C = US, ST = California, L = Pasadena, O = Example.com, OU = ITDept, CN =
    kmc.example.com, emailAddress = kmc_list@example.com
verify return:1
---
Certificate chain
 0 s:C = US, ST = California, L = Pasadena, O = Example.com, OU = ITDept, CN =
    kmc.example.com, emailAddress = kmc_list@example.com
  i:DC = com, DC = example, CN = Example.com Issuing Cert
-----BEGIN CERTIFICATE-----
MIIHFzCCBf+gAwIBAgITUwABBPeh10jhO4ybrAAAAEE9zANBgkqhkiG9w0BAQsF
```

[snip]

```
8LjUB9s3KvzgQ27MJDLvVWCTMWY5tyILmAcoRRYocKt/59VJGbWKZSdXgA==
-----END CERTIFICATE-----
 1 s:DC = gov, DC = nasa, DC = jpl, CN = JPLICA Issuing 1
  i:CN = JPLICA Root
-----BEGIN CERTIFICATE-----
MIIENDCCAxgyAwIBAgITPgAAAAALpoWeePlZ5cAAAAAAAjANBgkqhkiG9w0BAQsF
```

[snip]

```
vrufFb+w8AUV+1jcppRSIHsfTts8Jn4u
-----END CERTIFICATE-----
 2 s:CN = JPLICA Root
  i:CN = JPLICA Root
-----BEGIN CERTIFICATE-----
MIIDBzCCAe+gAwIBAgIQWv9RtOx18Y9Jdwc40lMSWTANBgkqhkiG9w0BAQsFADAW
```

[snip]

```

aHMXQHkUbBrMWto=
-----END CERTIFICATE-----
---
Server certificate
subject=C = US, ST = California, L = Pasadena, O = Example.com, OU = ITDept, CN =
    kmc.example.com, emailAddress = kmc_list@example.com

issuer=DC = com, DC = example, CN = Example.com Issuing Cert

---
Acceptable client certificate CA names
C = US, ST = California, L = Pasadena, O = Example.com, OU = test, CN = cmdenc-test-
    user.kmc.example.com, emailAddress = cmdenc-test-user@example.com
Requested Signature Algorithms:
    ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-
    PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-
    PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:ECDSA+SHA1:RSA
    +SHA1
Shared Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:RSA-
    PSS+SHA256:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-
    PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: ECDH, P-256, 256 bits
---
SSL handshake has read 4582 bytes and written 431 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
140109633697600:error:1409445C:SSL routines:ssl3_read_bytes:tlsv13 alert certificate
    required:ssl/record/rec_layer_s3.c:1544:SSL alert number 116

```

The key things in the above example to verify are that the Subject and Issuer for the certs presented match expectations, and down at the bottom, the ‘Verify Return Code: 0 (ok)’ line states that all the certificate validation succeeded.

See Section 4.1 for more details on how to generate, sign, and otherwise prepare TLS artifacts for use with the DCS Service and Client.

6 ADMINISTRATION

6.1 SECURITY ASSOCIATION (SA) MANAGEMENT

The SDLS capabilities in DCS use "Security Associations (SAs)" to control how SDLS is applied to TC transfer frames (and SDLS is reversed). The SAs are records in a Security Association Database (SADB), as described in Section 4.2.4.

The SAs in the SADB can be managed by using the `kmc-sa-mgmt` command line interface (CLI) described in Section 6.1.1, or by using SA Management GUI described in Section 6.1.2.

6.1.1 SA MANAGEMENT CLI

Before using the `kmc-sa-mgmt` CLI, configure the `kmc-sa-mgmt.properties` file as described in Section 5.2.3.1.

The `kmc-sa-mgmt` CLI provides subcommands for SA management functions, as shown below.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt --help
```

```
Usage: kmc-sa-mgmt [-hV] [COMMAND]
KMC Security Association Management CLI
  -h, --help          Show this help message and exit.
  -V, --version       Print version information and exit.

Commands:
  list      List security associations
  create    Create a new Security Association
  update    Update an existing Security Association
  delete    Delete a Security Association
  key       Key or rekey a Security Association
  start     Start a Security Association. Only one (1) SA can be active per (SPI,
           GVCID) permutation
  stop      Stop a Security Association
  expire    Expire a Security Association
```

The `kmc-sa-mgmt` CLI can be used to list SAs in the SADB, as shown below.

```
$ /opt/amos/kmc/bin/kmc-sa-mgmt list --help
```

```
Usage: kmc-sa-mgmt list [-hV] [--scid=<scid>] [--spi=<spi>] [-e | --mysql |
           --json] [--active | --inactive]
List security associations
  --active          Only list active SAs
  -e, --extended    extended CSV format
  -h, --help        Show this help message and exit.
  --inactive        Only list inactive SAs
  --json           JSON format
  --mysql          MySQL-like key-value format
  --scid=<scid>     Filter by spacecraft ID
  --spi=<spi>       Filter by security parameter index
  -V, --version     Print version information and exit.
```

Notes:

1. The `-e` (a.k.a., `--extended`) option can be used to get an output that can be saved to a CSV file to use as input to bulk creation/updating of SAs.
2. The `-e` (`--extended`), `--json`, and `--mysql` options provide different output formats and are mutually exclusive.

The kmc-sa-mgmt CLI can be used to create SAs in the SADB, as shown below.

```
$ /opt/ammios/kmc/bin/kmc-sa-mgmt create --help
```

```
Usage: kmc-sa-mgmt create [-hV] ([[--tfvn=<tfvn> --scid=<scid> --vcid=<vcid>
--mapid=<mapid> [--spi=<spi>]] [--ekid=<ekid>
--ecs=<ecs>] [--akid=<akid>] [--acs=<acs>]
--iv=<iv>] [--ivlen=<ivLen>] [--arsn=<arsn>]
--arsnlen=<arsnlen>] [--arsnw=<arsnw>] [--abm=<abm>]
--abmlen=<abmLen>] [--shivflen=<shivflen>]
--shsnflen=<shsnflen>] [--shplflen=<shplflen>]
--stmacflen=<stmacflen>] [--st=<st>]]] |
--file=<file>])
Create a new Security Association
--abm=<abm>          authentication bitmask (hex string)
--abmlen=<abmLen>    authentication bitmask length (bytes)
--acs=<acs>          authentication cypher suite
--akid=<akid>         authentication key ID
--arsn=<arsn>        anti-replay sequence number starting value (hex
                    string)
--arsnlen=<arsnlen>  anti-replay sequence number length (bytes)
--arsnw=<arsnw>      anti-replay sequence number window
--ecs=<ecs>          encryption cypher suite (hex string)
--ekid=<ekid>        encryption key ID
--file=<file>
-h, --help          Show this help message and exit.
--iv=<iv>           initialization vector (hex string)
--ivlen=<ivLen>     initialization vector length, in bytes
--mapid=<mapid>      multiplexer access ID
--scid=<scid>       spacecraft ID
--shivflen=<shivflen> security header initialization vector field length
                    (bytes)
--shplflen=<shplflen> security header pad length field length (bytes)
--shsnflen=<shsnflen> security header sequence number field length (bytes)
--spi=<spi>         security parameter index
--st=<st>           service type. 0 = PLAINTEXT, 1 = ENCRYPTION, 2 =
                    AUTHENTICATION, 3 = AUTHENTICATED_ENCRYPTION
--stmacflen=<stmacflen> security trailer MAC field length (bytes)
--tfvn=<tfvn>       transfer frame version number
-V, --version       Print version information and exit.
--vcid=<vcid>       virtual channel ID
```

The kmc-sa-mgmt CLI can be used to update SAs in the SADB. When updating an SA, only the parameters that are being updated need to be specified. If a CSV file is used as input for bulk updates, existing SAs will be updated but new SAs will not be created (use the "create" subcommand to create new SAs).

The usage statement for updating SAs is shown below.

```
$ /opt/ammios/kmc/bin/kmc-sa-mgmt update --help
```

```
Usage: kmc-sa-mgmt update [-hV] ([[--tfvn=<tfvn> --scid=<scid>
--vcid=<vcid>] [--mapid=<mapid> --spi=<spi>]
[--ekid=<ekid>] [--ecs=<ecs>] [--akid=<akid>]
--acs=<acs>] [--iv=<iv>] [--ivlen=<ivLen>]
--arsn=<arsn>] [--arsnlen=<arsnlen>]
--arsnw=<arsnw>] [--abm=<abm>] [--abmlen=<abmLen>]
--shivflen=<shivflen>] [--shsnflen=<shsnflen>]
--shplflen=<shplflen>] [--stmacflen=<stmacflen>]
--st=<st>]]] | [--file=<file>])
Update an existing Security Association
--abm=<abm>          authentication bitmask (hex string)
```



```

--abmlen=<abmlen>      authentication bitmask length (bytes)
--acs=<acs>             authentication cypher suite
--akid=<akid>           authentication key ID
--arsn=<arsn>           anti-replay sequence number starting value (hex
                        string)
--arsnlen=<arsnlen>     anti-replay sequence number length (bytes)
--arsnw=<arsnw>         anti-replay sequence number window
--ecs=<ecs>             encryption cypher suite (hex string)
--ekid=<ekid>           encryption key ID
--file=<file>
-h, --help             Show this help message and exit.
--iv=<iv>              initialization vector (hex string)
--ivlen=<ivlen>         initialization vector length, in bytes
--mapid=<mapid>         multiplexer access ID
--scid=<scid>           spacecraft ID
--shivflen=<shivflen>  security header initialization vector field length
                        (bytes)
--shplflen=<shplflen>  security header pad length field length (bytes)
--shsnflen=<shsnflen>  security header sequence number field length (bytes)
--spi=<spi>             security parameter index
--st=<st>              service type. 0 = PLAINTEXT, 1 = ENCRYPTION, 2 =
                        AUTHENTICATION, 3 = AUTHENTICATED_ENCRYPTION
--stmacflen=<stmacflen> security trailer MAC field length (bytes)
--tfvn=<tfvn>          transfer frame version number
-V, --version          Print version information and exit.
--vcid=<vcid>          virtual channel ID

```

For example, to reset the ICV to a value of "...01" in SPI 123 for SCID 44, use a command like following (with the correct values for your system):

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt update --spi=123 --scid=44 \
--icv= X'00000000000000000000000000000001'
```

Note: Both the SPI and SCID must be provided because DCS can support using the same SPI for different SCIDs.

The kmc-sa-mgmt CLI can be used to delete SAs, as shown in the usage statement below.

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt delete --help
```

```
Usage: kmc-sa-mgmt delete [-hVy] --scid=<scid> --spi=<spi> [--spi=<spi>]...
```

Delete a Security Association

```

-h, --help             Show this help message and exit.
--scid=<scid>          spacecraft id
--spi=<spi>             security parameter index
-V, --version          Print version information and exit.
-y                    Delete without confirmation

```

The kmc-sa-mgmt CLI can be used to set keys for SAs, as shown in the usage statement below.

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt key --help
```

```
Usage: kmc-sa-mgmt key [-hV] --scid=<scid> --spi=<spi> [--ekid=<ekid>]
                        [--ecs=<ecs>] [--akid=<akid>] [--acs=<acs>] [-y]
```

Key or rekey a Security Association

```

--acs=<acs>             authentication cypher suite
--akid=<akid>           authentication key ID
--ecs=<ecs>             encryption cypher suite (hex string)
--ekid=<ekid>           encryption key ID
-h, --help             Show this help message and exit.
--scid=<scid>          spacecraft id
--spi=<spi>             security parameter index
-V, --version          Print version information and exit.
-y                    Rekey without confirmation

```

The `kmc-sa-mgmt` CLI can be used to start an SAs, as shown in the usage statement below.

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt start --help
Usage: kmc-sa-mgmt start [-fhV] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Start a Security Association. Only one (1) SA can be active per (SPI, GVCID)
permutation
  -f, --force          Force an SA to start, if another SA is currently active.
  -h, --help          Show this help message and exit.
                     --scid=<scid>      spacecraft ID
                     --spi=<spi>       security parameter index
  -V, --version        Print version information and exit.
```

The `kmc-sa-mgmt` CLI can be used to stop an SA, as shown in the usage statement below.

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt stop --help
Usage: kmc-sa-mgmt stop [-fhV] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Stop a Security Association
  -f, --force          Force an SA to start, if another SA is currently active.
  -h, --help          Show this help message and exit.
                     --scid=<scid>      spacecraft ID
                     --spi=<spi>       security parameter index
  -V, --version        Print version information and exit.
```

The `kmc-sa-mgmt` CLI can be used to expire an SA, as shown in the usage statement below.

```
$ /opt/amm0s/kmc/bin/kmc-sa-mgmt expire --help
Usage: kmc-sa-mgmt expire [-hVy] --scid=<scid> --spi=<spi> [--spi=<spi>]...
Expire a Security Association
  -h, --help          Show this help message and exit.
                     --scid=<scid>      Spacecraft id
                     --spi=<spi>       Security parameter index
  -V, --version        Print version information and exit.
  -y                  Expire without confirmation
```

Expiring an SA will clear the `ekid/akid` (i.e., the key reference) and set `sa_state` to 1 (unkeyed). To re-use an SA that has been expired, use the "key" subcommand to set the key, and then the "start" subcommand (when ready to start the SA).

6.1.2 AMMOS SA MANAGEMENT GUI

DCS provides a Web Graphical User Interface (GUI) for managing the Security Associations (SAs) used by command and telemetry encryption. To access the AMMOS SA Management GUI, open a Web browser and go to the following URL:

`https://<DCS server FQDN>:<port>/sa-mgmt`

(for example: `https://kmc-server.example.com:8447/sa-mgmt`)

Figure 2 below shows the AMMOS SA Management GUI.

AMMOS Security Association Database (SADB) Management

COLUMNS

DELETE

+ CREATE

BULK

EXPORT

RELOAD

	SPI	SCID	VCID	TFVN	MAPID	State	Service Type	EKID	AKID	ECS	ACS
■	1	44	0	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		AES256-GCM	None
■	2	44	1	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		AES256-GCM	None
■	3	44	2	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		AES256-GCM	None
■	4	44	3	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		AES256-GCM	None
■	5	44	7	0	0	Operational	AUTHENTICATION		kmc/test/key130	None	AES256-CMAC
■	6	44	8	0	0	Operational	AUTHENTICATION		kmc/test/nist_hmacsha256	None	HMAC-SHA256
■	7	44	9	0	0	Operational	AUTHENTICATION		kmc/test/nist_hmacsha512	None	HMAC-SHA512
■	8	44	10	0	0	Operational	AUTHENTICATION		kmc/test/nist_hmacsha512	None	HMAC-SHA512
■	9	44	11	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		AES256-GCM	None

Figure 2: AMMOS SA Management GUI

The GUI can be used to create individual SAs, bulk upload SAs from a CSV file (which will create SAs that do not exist yet, and will modify existing SAs to match the uploaded file), individually edit SAs, delete one or more SAs, and export SAs to a CSV file. The instructions in this section describe how to perform those functions in the GUI, and how to adjust the set of displayed columns, filter the set of displayed SAs, and reload the content shown in the GUI.

The set of columns displayed in the GUI can be adjusted by clicking on the COLUMNS button at the upper-left of the GUI. The window shown in Figure 3 will appear. Select/deselect the columns names to adjust which columns will be displayed. Click anywhere else in the GUI to dismiss the window shown in Figure 3. The RESET button will return the set of selected columns back to the default set.

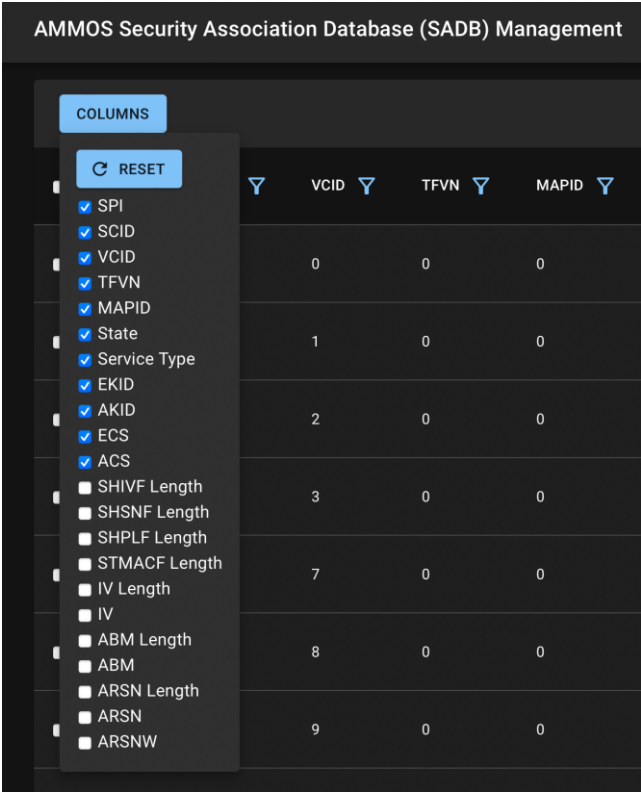


Figure 3: SADB Management GUI Column Selection

To filter the set of displayed SAs, click on a filter icon and enter a value, as shown in Figure 4. Multiple filters can be used at the same time. Click the X next to enabled filters to clear them.

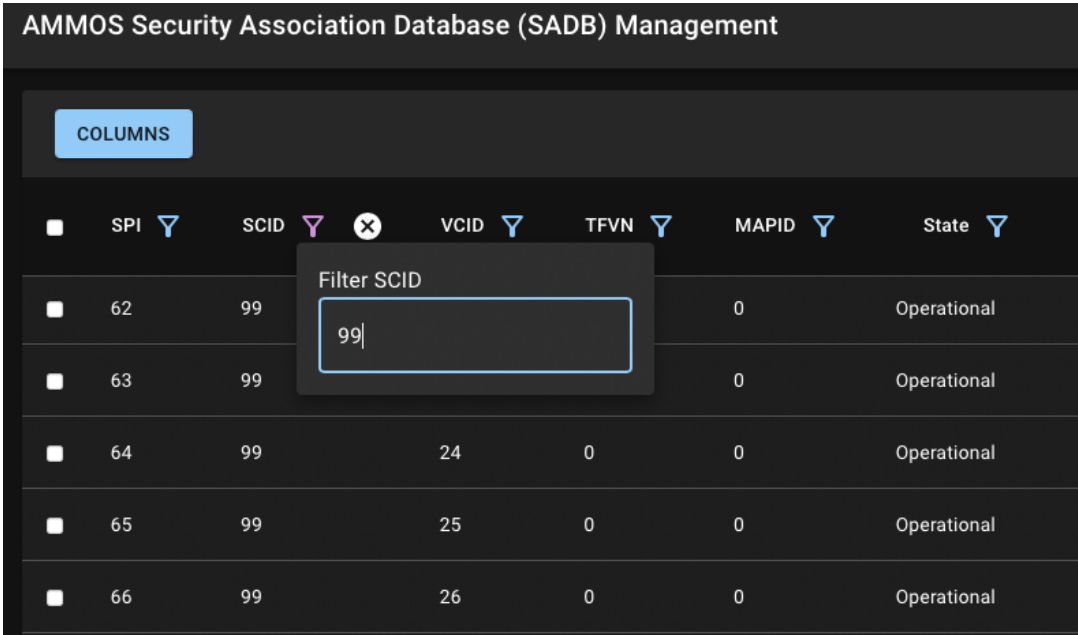


Figure 4: SA Management GUI - Filtering Displayed SAs

To create an individual SA, click the "+ CREATE" button at the top of the SAB Management GUI main window. The window shown in Figure 5 will appear. There are several parameters in an SA. Scroll down to see them all. Figure 6 shows the bottom half of the parameters, shown after scrolling down.

Create SA

SPI
0

Security Parameter Index

SCID
0

Spacecraft ID

VCID
0

Virtual Channel ID

TFVN
0

Transfer Frame Version Number

MAPID
0

Multiplexer Access Point ID

State
Unkeyed | Expired

SA State: Unkeyed or Expired = 1, Keyed = 2, Operational = 3

Service Type
PLAINTEXT

Service Type: 0 = PLAINTEXT, 1 = AUTHENTICATION, 2 = ENCRYPTION, 3 = AUTHENTICATED_ENCRYPTION

EKID

Encryption Key ID

AKID

Authentication Key ID

ECS
AES256-GCM

Encryption Cipher Suite: None = 0, AES256-GCM = 1

ACS
None

Authentication Cipher Suite: None = 0, AES256-CMAC = 1, HMAC-SHA256 = 2, HMAC-SHA512 = 3

SHIVF Length
0

CREATE CANCEL

Figure 5: SA Management GUI - Create SA (top)

Create SA

Security Header Initialization Vector Field Length

SHSNF Length

0

Security Header SN7 Field Length

SHPLF Length

0

Security Header PL Field Length

STMACF Length

0

Security Trailer MAC Field Length

IV Length

12

Initialization Vector Length

IV

0x 00000000000000000000000000000000

Initialization Vector

ABM Length

19

Application Binary Mask Length

ABM

0000FC0000FFFF0000000000000000000000000000

0x

Application Binary Mask

ARSN Length

20

Anti-Replay Sequence Number Length

ARSN

0x 00

Anti-Replay Sequence Number

ARSNW

0

Anti-Replay Sequence Number Window

CREATE CANCEL

Figure 6: SA Management GUI - Create SA (bottom)

Enter the appropriate values for the SA. At least enter the SPI and SCID, which cannot be edited after the SA is created. If an SA is created with the wrong SPI/SCID, simply delete the incorrect SA and create a new one with the correct values.

See Section 4.2.4 for definitions of the SA parameters.

Please note that the 0x portion of hex values is already indicated in the GUI, and should not be included in the hex values entered into the GUI.

The SA Management GUI will perform some checking of the inputs after the CREATE button is clicked. In 3.7.0 and prior DCS releases, the GUI cannot create or update SAs with a null IV. This limitation will be addressed in a future version of DCS.

To edit an SA individually (including actions to start, stop, expire, change the IV/ARSN, or rekey the SA), simply click on the row of that SA. The Edit SA window shown in Figure 7 will appear.

Edit SA

DELETE STOP EXPIRE IV/ARSN (RE)KEY

SPI
3

Security Parameter Index

SCID
44

Spacecraft ID

VCID
2

Virtual Channel ID

TFCN
0

Transfer Frame Version Number

MAPIID
0

Multiplexer Access Point ID

State
Operational

SA State: Unkeyed or Expired = 1, Keyed = 2, Operational = 3

Service Type
AUTHENTICATED_ENCRYPTION

Service Type: 0 = PLAINTEXT, 1 = AUTHENTICATION, 2 = ENCRYPTION, 3 = AUTHENTICATED_ENCRYPTION

EKID
kmc/test/key130

Encryption Key ID

AKID

Authentication Key ID

ECS
AES256-GCM

Encryption Cipher Suite: None = 0, AES256-GCM = 1

ACS
None

Authentication Cipher Suite: None = 0, AES256-CMAC = 1, HMAC-SHA256 = 2, HMAC-SHA512 = 3

UPDATE CANCEL

Figure 7: SA Management GUI - Edit SA

The DELETE button shown in Figure 7 can be used to delete the SA. A confirmation dialog box (not shown) will appear. Click the CONTINUE or CANCEL button, as appropriate.

The SA shown in Figure 7 is already started, and thus a STOP button is shown. Clicking the STOP button will cause a confirmation dialog box to appear, as shown in the figure below.

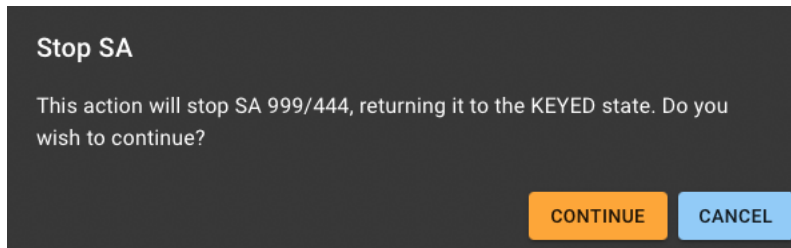


Figure 8: SA Management GUI - Stop SA Confirmation

Clicking the CONTINUE button in the Stop SA dialog box will Stop the SA (i.e., change the State of the SA to Keyed in the SADB) and update the Edit SA window to reflect the changes. Clicking the CANCEL button will dismiss the Stop SA dialog box without making any changes. It is not necessary to click the UPDATE button in the Edit SA window (i.e., because clicking CONTINUE already updated the SADB) unless other changes are made.

If an SA is not in the Operational (i.e., started/active) State, a START button will be shown instead of the STOP button. Clicking the START button will cause a confirmation dialog box shown in Figure 9 to appear.

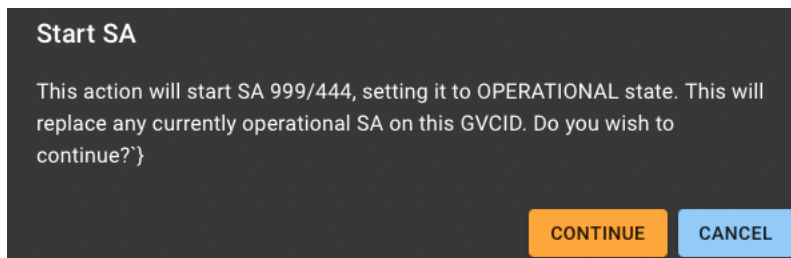


Figure 9: SA Management GUI - Start SA Confirmation

Clicking the CONTINUE button will Start the SA (in the SADB), stop any other SAs for the same GVCID, and update the Edit SA window to reflect the changes. Clicking the CANCEL button in the Start SA dialog box will dismiss it without making any changes. It is not necessary to click the UPDATE button in the Edit SA window (i.e., because clicking CONTINUE already updated the SADB) unless other changes are made.

To expire the SA (i.e., set it back to the "Unkeyed | Expired" State), click the EXPIRE button. The Expire SA confirmation dialog box shown in Figure 10 will appear.

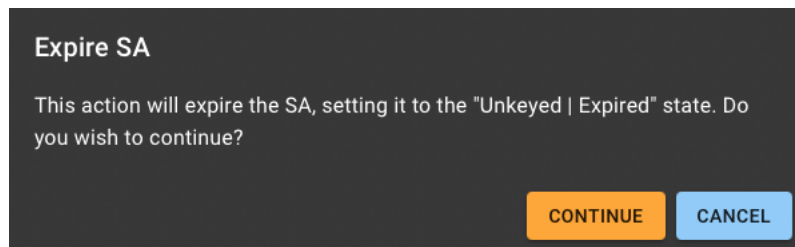


Figure 10: SA Management GUI - Expire SA Confirmation

Clicking the CONTINUE button will expire the SA and erase the ekid or akid value in the SADB, and update the Edit SA window to reflect the changes. Click CANCEL to dismiss the Expire SA dialog box without making any changes. It is not necessary to click the UPDATE button in the Edit SA window (i.e., because clicking CONTINUE already updated the SADB) unless other changes are made.

The IV and ARSN fields can be edited directly in the Edit SA window. There is an IV/ARSN button that can be clicked to display the Reset IV and/or ARSN window shown in Figure 11. However, that window will not correctly update the value for IV or ARSN in the Edit SA window. The CANCEL button can be used to avoid making changes.

 A dark-themed window titled "Reset IV and/or ARSN". The text inside reads: "Reset IV or ARSN to 0, or a specific value." Below this are several input fields:

- IV**: A text box containing "0x 000000000000000000000001".
- Initialization Vector**: A label above the IV field.
- IV Length**: A text box containing "12".
- IV Length in bytes**: A label above the IV Length field.
- ARSN**: A text box containing "0x".
- Anti-Replay Sequence Number**: A label above the ARSN field.
- ARSN Length**: A text box containing "0".
- ARSN length in bytes**: A label above the ARSN Length field.
- ARSNW**: A text box containing "5".
- ARSN Window**: A label above the ARSNW field.

 At the bottom right, there are two buttons: "CONTINUE" (grey) and "CANCEL" (blue).

Figure 11: SA Management GUI - Reset IV/ARSN

The Encryption Key Identifier (ekid) and Authentication Key Identifier (akid) entries can be edited directly in the Edit SA window, or the REKEY button can be clicked to display the window shown in Figure 12.

Figure 12: SA Management GUI - Rekey

Clicking the CONTINUE button in the (Re)key SA window will update the ekid or akid value in the SADB and update the Edit SA window to reflect the changes. Clicking the CANCEL button will dismiss the Re(key) SA dialog box without making any changes. It is not necessary to click the UPDATE button in the Edit SA window (i.e., because clicking CONTINUE already updated the SADB) unless other changes are made.

To delete one or more SAs, use the checkboxes on the left-hand side to select the SA(s) to be deleted, as shown for SAs 222 and 223 in Figure 13. Then, click the DELETE button (shown circled in red).

	SPI	SCID	VCID	TFVN	MAPID	State	Service Type
☐	22	44	22	0	0	Operational	AUTHENTICATED_ENCRYPTION
☒	222	88	1	0	0	Operational	AUTHENTICATED_ENCRYPTION
☒	223	88	1	0	0	Unkeyed Expired	PLAINTEXT

Figure 13: SA Management GUI - Delete SAs

After clicking the DELETE button, the confirmation dialog box shown in Figure 14 will appear. Verify the right SA(s) will be deleted. Click the CONTINUE button to proceed with the deletion, or click the CANCEL button to cancel the deletion.

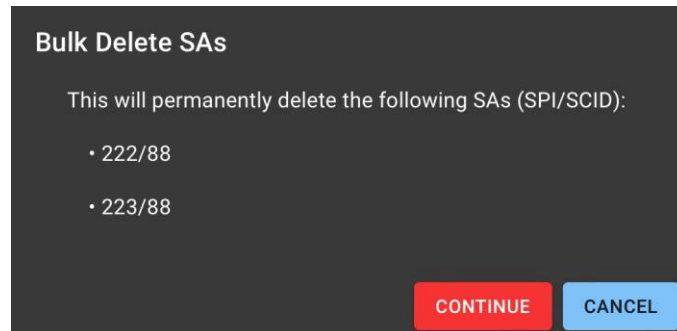


Figure 14: SA Management GUI - Confirm SA Deletion

After the SA(s) is(are) deleted, a green status indicator will appear briefly at the lower-left side of the GUI, as shown in Figure 15.

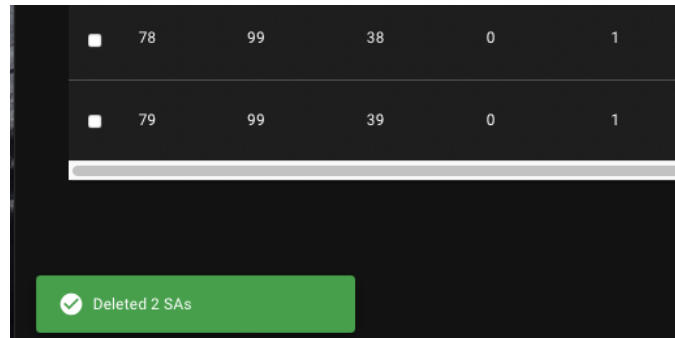


Figure 15: SA Management GUI - SAs Deleted

To bulk create new SAs, click the BULK button, circled in red in the figure below.

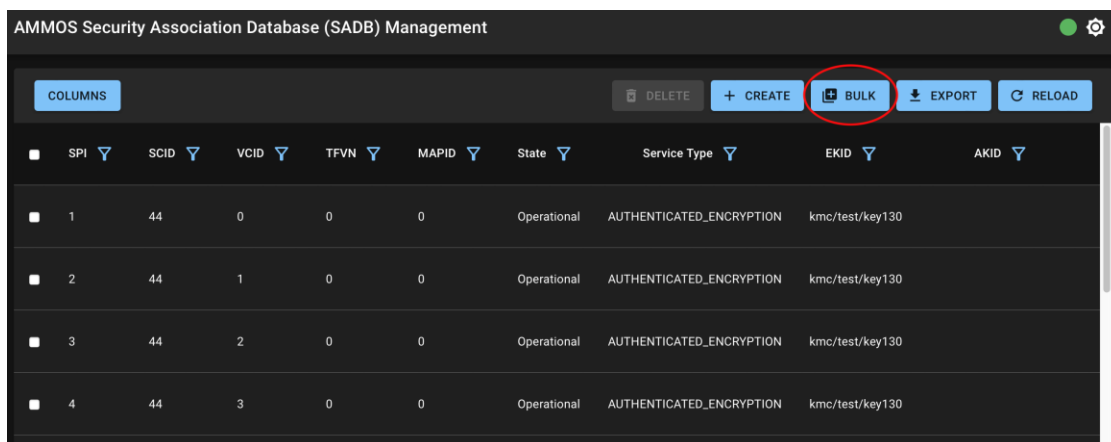


Figure 16: Choose Bulk Load of SAs

The figure shown below will appear.

Figure 17: SA Management GUI - Select CSV File

Click the Choose File button, and select a CSV file to import to create SAs. After selecting a file, the filename will appear at the bottom of the dialog box, as shown in Figure 18. Click the CONTINUE button to import the file in order to create SAs from the data in the file. The ESC key can be clicked to dismiss the dialog box without importing a file.

Figure 18: SA Management GUI - CSV File Selected

To export the contents of the SADB to a CSV file, click the EXPORT button circled in red in Figure 19.

	SPI	SCID	VCID	TFVN	MAPID	State	Service Type	EKID	AKID
1	44	0	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		
2	44	1	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		
3	44	2	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		
4	44	3	0	0	Operational	AUTHENTICATED_ENCRYPTION	kmc/test/key130		

Figure 19: SA Management GUI - Export CSV

To reload the GUI from the SADB, click the RELOAD button circled in red in Figure 20

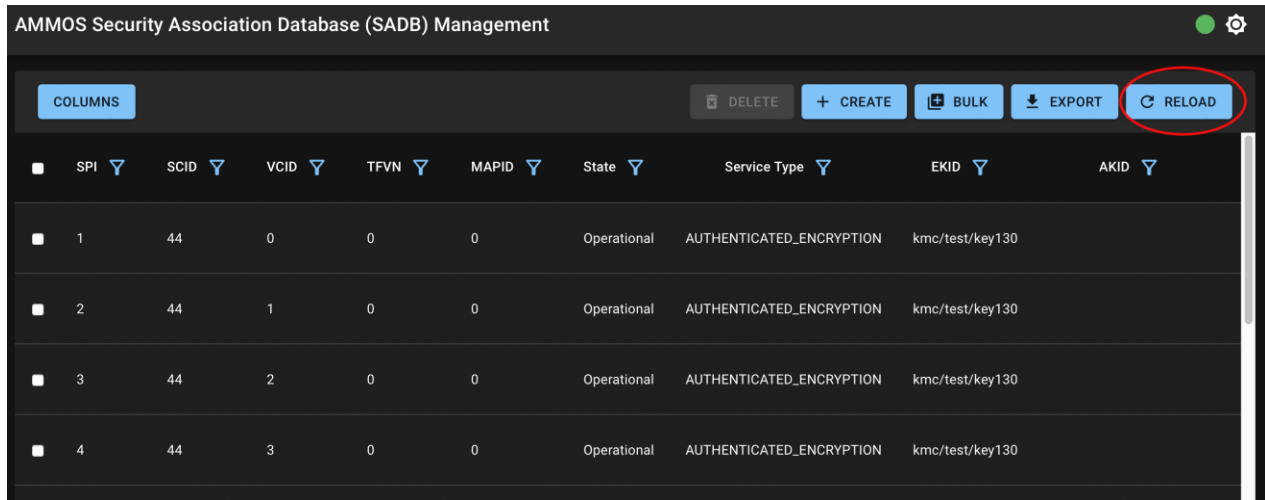


Figure 20: SA Management GUI - Reload

6.2 LOGGING

There are no administrative actions specific to DCS logging other than reviewing and managing the log files generated by the DCS software.

6.2.1 CRYPTO CLIENT LOGS

The DCS Crypto Client generates log file entries when the Crypto Client is used. The log files can be used for troubleshooting and should be reviewed periodically for signs of problems.

The DCS Crypto Client comes with two categories of logging: debugging and audit. The files associated with these logs can be found in the `/opt/ammos/kmc/logs/` directory.

The DCS logging system can be set to four different levels of verbosity :

`info`, `debug`, `error` and `trace`.

For log4j 2.x the logging verbosity level can be set by changing the ‘Logger name’ entry parameter *level* from “info” to “debug” in `/opt/ammos/kmc/etc/kmc-crypto-log4j2.xml` configuration file.

Log file rotation is set up by default (in `/opt/ammos/kmc/etc/kmc-crypto-log4j2.xml`) to limit DCS Crypto Client log files to a maximum of 100MB and to keep the last 30 files before they are overwritten. Depending on the project's requirements for log data retention, system administrators may need to archive DCS log files before they are overwritten.

The `/opt/ammos/kmc/logs/` directory must be readable/writable by all users that use applications that use the DCS Crypto Client. If the Crypto Client has been used, but the DCS Crypto Client log files are missing or empty, make sure the group/permissions are correct and that the users are in the correct group. See the Crypto Client installation instructions in Section 5.2.3.1 for details about how to set up access to DCS logs during DCS deployment.

6.2.2 CRYPTO SERVICE LOGS

The DCS Crypto Service (i.e., Crypto HTTPS API) generates audit and debug log file entries when the Crypto Service is used. The log files can be used for troubleshooting purposes and should be reviewed periodically to look for signs of problems.

The Crypto Service logs are located at `/opt/amos/kmc/services/crypto-service/logs`.

Log file rotation is set up by default (in `/opt/amos/kmc/services/crypto-service/etc/kmc-crypto-service-log4j2.xml`) to limit DCS Crypto Client log files to a maximum of 100MB and to keep the last 30 files before they are overwritten. Depending on the project's requirements for log data retention, system administrators may need to archive DCS log files before they are overwritten.

Crypto Service administrators must be members of the `crypto` group in order to be able to read the Crypto Service log files. The `crypto` group is created on the local host if that group does not already exist. As noted in Section 2.5, creating the `crypto` group in LDAP (e.g., in the Projects branch, for JPL missions) facilitates managing operating system groups by using a directory service rather than on a host-by-host basis.

6.2.3 SDLS SERVICE LOGS

Logging for SDLS Service is configured in the `/opt/amos/kmc/services/sdls-service/etc/kmc-sdls-service-log4j2.xml` configuration file.

Log messages for the SDLS Service are written to the `/opt/amos/kmc/services/sdls-service/logs/kmc-sdls-server.log` log file.

SDLS Service administrators must be members of the `crypto` group in order to be able to read the Crypto Service log files.

6.2.4 SA MANAGEMENT GUI LOGS

Logging for the SA Management GUI is configured in the `/opt/amos/kmc/services/sa-mgmt/etc/kmc-sa-mgmt-log4j2.xml` configuration file.

Log messages for the SA Management GUI are written to the `/opt/amos/kmc/services/sa-mgmt/logs/kmc-sa-mgmt.log` log file.

SA Management GUI administrators must be members of the `crypto` group to be able to read the SA Management GUI log files.

6.3 ENVIRONMENT VARIABLES

The environment variables in the table below are used as a reference when the DCS Crypto Library is installed in a directory other than the default locations. While the variables in the table below will automatically set via the installation process, it is advised to also set these variables via the appropriate `.rc` file (ie, `.bashrc`, `.cshrc`, etc).

Table 5: Environment Variables

Variable Name	Default Value	Description
KMC_HOME	/opt/ammos/kmc	Reference to DCS base directory.

6.4 NETWORK INTERFACES

Table 6: TCP/UDP Port Information

Port Type	Default Port #	Description
TCP/HTTPS	8443	DCS Crypto Service port
TCP/HTTPS	8445	DCS SDLS Service port
TCP/HTTPS	8447	DCS SA Management GUI Service port
TCP/TLS	3306	SADB (MariaDB) port

Note: The TCP ports used by the DCS Services and SADB can be changed via configuration.

7 PRODUCT SUPPORT

7.1 DEFECT REPORTING PROCEDURE

Send an email to the Problem Reporting email address shown in Section 6.2. Please include point of contact information, a description of the problem, and a description of the conditions when the problem occurred (e.g., the DCS capability being exercised, operating system version, DCS version, and other relevant details). A DCS development task member will respond to collect any additional information that is needed and to provide technical assistance.

Flight projects at JPL should report issues, and make feature requests, via the AMMOS Service Desk (<https://jira.jpl.nasa.gov/servicedesk/customer/portal/40>).

7.2 POINTS OF CONTACT

Table 7: Points of Contact

Type	Point of Contact
Problem Reporting	mgss.css.problems@jpl.nasa.gov
Adaptation	mgss.css.requests@jpl.nasa.gov
General Help	mgss.css.requests@jpl.nasa.gov

APPENDIX A - ABBREVIATIONS AND ACRONYMS

AES	Advanced Encryption Standard
AMMOS	Advanced Multi-Mission Operations System
ASEC	Application Security
BCFIPS	BouncyCastle FIPS 140-2 Certified Crypto Library
BCFKS	BouncyCastle FIPS Keystore
CAM	Common Access Manager
CCSDS	Consultative Committee for Space Data Systems
CM	Configuration Management
CMAC	Cipher-based Message Authentication Code
DES	Data Encryption Standard
DESede	Triple DES (Encrypt-Decrypt-Encrypt)
FIPS	Federal Information Processing Standard
GCM	Galois/Counter Mode
HMAC	Hash-based Message Authentication Code
ICV	Integrity Check Value
IT	Information Technology
JCE	Java Cryptography Extension
JPL	Jet Propulsion Laboratory
KMIP	Key Management Interoperability Protocol
KMS	Key Management Service
MGSS	Multimission Ground Systems & Services
NASA	National Aeronautics and Space Administration
OCI	Open Container Initiative
Rev	Revision
RP	Release Plan
RSA	Rivest -Shamir-Adleman asymmetric cryptography algorithm
SDD	Software Design Document
SDLS	Space Data Link Security
SE	Systems Engineer

SEO	Systems Engineering Office
SHA	Secure Hash Algorithm
URL	Uniform Resource Locator

* Note: The file contains est and ast columns, which are used internally and not setable via the kmc-sa-mgmt CLI (i.e., the tools ignores all unknown columns). Those fields are removed from the example below.

- SA 1 is Operational, AES-GCM, 12-byte IV as counter, VCID and SPI are authenticated (along with ciphertext of payload), on VCID 0
- SA 2 is Operational, AES-GCM, 12-byte IV as counter, all primary header fields and SPI are authenticated (along with ciphertext of payload), on VCID 1
- SA 3 is Keyed, AES-GCM, 12-byte IV as counter, all primary header fields and SPI are authenticated (along with ciphertext of payload), on VCID 1
- SA 4 is Operational, AES-CMAC, 4-byte Seq # counter, VCID/SPI/payload authenticated, on VCID 2
- SA 5 is Keyed, AES-CMAC, 4-byte Seq # counter, VCID/SPI/payload authenticated, on VCID 2
- SA 6 is Operational, AES-CMAC, 4-byte Seq # counter, whole frame authenticated, on VCID 3
- SA 7 is Keyed, AES-CMAC, 4-byte Seq # counter, whole frame authenticated, on VCID 3

85

